ASHESI UNIVERSITY

Customer Segmentation Using Machine Learning for Small and Medium Ghanaian Enterprises

APPLIED PROJECT

B.Sc Management Information Systems

Akua Birago Oduro

2026

ASHESI UNIVERSITY COLLEGE

Customer Segmentation Using Machine Learning for Small and Medium Ghanaian Enterprises

APPLIED PROJECT

Applied Project submitted to the Department of Computer Science, Ashesi University College in partial fulfilment of the requirements for the award of Bachelor of Science degree in Management Information Systems.

Akua Birago Oduro

2026

DECLARATION

I hereby declare that this Applied Project is the result of my own original work and that no part of it has been presented for another degree in this university or elsewhere.

Candidate's Signature: ……

Candidate's Name: Akua Birago Oduro

Date: 7th April 2026 .

I hereby declare that preparation and presentation of this Applied Project was supervised in accordance with the guidelines on supervision of Applied Project laid down by Ashesi University.

Supervisor's Signature: ……

Supervisor's Name: Professor Robert Sowah

Date: 7th April 2026.

Acknowledgments

I would like to first thank God for granting me the strength, wisdom, and perseverance to complete this project.

I am sincerely grateful to my supervisor, Professor Robert Sowah, for his guidance, patience, and continuous encouragement throughout this work. His detailed feedback, technical advice, and willingness to share resources greatly shaped both my analysis and the overall development of this project. His constant push for improvement has been invaluable.

I would also like to acknowledge the SME participants who contributed their time and insights, which were essential to the success of this study.

Finally, I am deeply grateful to my family for their unwavering encouragement, support, and prayers throughout this journey

Abstract

Small and medium-sized enterprises make up over 90% of registered businesses in Ghana, yet most rely on intuition, informal records, and basic records for customer and marketing decisions. This limits their ability to identify high-value customers, detect early signs of customer loss, or tailor marketing strategies and promotions to different customer groups. This project develops Customer360, a web-based customer segmentation platform that helps Ghanaian SMEs turn transaction records into practical customer groups and actionable business insights and recommendations.

The system uses a guided CSV-based workflow designed for non-technical users. Users upload a dataset, preview the detected columns, map the required fields, and validate parsing settings before analysis begins. The backend cleans the data, computes customer-level Recency, Frequency, and Monetary features, compares K-Means, Gaussian Mixture Model, and Hierarchical Clustering, and selects the best-performing method. Results are presented through a browser-based dashboard with plain-language segment labels, chart summaries, a segmented CSV export, and a generated PDF report. The system runs on an Ubuntu server with a PHP web layer proxying requests to a FastAPI analytics backend, and MySQL storing user and job metadata.

By combining guided data validation, interpretable machine learning, plain-language reporting, and a live deployed workflow, Customer360 demonstrates that data-driven customer analysis can be made more accessible to smaller businesses operating under real-world constraints The deployed system is available at:

http://64.23.248.187:8080/customer360/

Keywords: Customer segmentation, Ghanaian SMEs, RFM analysis, machine learning, clustering, K-Means, Gaussian Mixture Model, Hierarchical Clustering, explainable AI, SHAP, business intelligence, decision support system, customer analytics, web-based analytics platform.

Table Of Contents

DECLARATION ..... **i**

Abstract ..... **iii**

Chapter 1: Introduction **1**

1.1 Background **1**

1.2 Problem Statement and Justification **3**

1.3 Aims of the Project **4**

1.3.1 OBJ 1: Data Collection and Preparation 4

1.3.2 OBJ 2: Model Development and Deployment 5

1.3.3 OBJ 3: Evaluation and Validation 5

1.3.4 OBJ 4: Explainability 5

1.3.5 OBJ 5: System Integration and Recommendations 5

1.4 Proposed Solution Approach **6**

1.4.1 Component A: Data Ingestion and Validation Module. 6

1.4.2 Component B: Data Processing and Segmentation Engine. 7

1.4.3 Component C: Explainability and Visualization Module. 7

1.4.4 Component D: Automated Reporting Module. 8

1.5 Relevance of the Project **8**

1.6 Conceptual Definitions **10**

1.6.1 Customer Segmentation 10

1.6.2 The RFM Model 10

1.6.3 Unsupervised Clustering Algorithms 11

1.6.4 Cluster Evaluation Metrics 11

1.6.5 Explainable AI and SHAP 12

1.7 Empirical Issues **12**

1.7.1 Performance of Clustering Algorithms in Customer Segmentation 12

1.7.2 Limitations of the RFM Model 13

1.7.3 XAI Adoption in Customer Analytics 13

1.7.4 Data Quality and Analytics Challenges for African SMEs 14

1.8 Research Gaps **15**

1.9 Conceptual Framework and Comparative Analysis **16**

1.9.1 RFM as the Feature Engineering Approach 16

1.9.2 Multi-Algorithm Clustering with Majority-Vote K Selection 16

1.9.3 SHAP Surrogate Model for Explainability 17

1.10 Comparative Analysis of Technologies **18**

1.10.1 Organization of Report **19**

Chapter 2: Requirements 21

2.1 Introduction **21**

2.2 System Overview **21**

2.3 Project Management Methodology **22**

2.4 Requirement Gathering **23**

2.4.1 Literature and related-system review 23

2.4.2 Stakeholder interviews 23

2.4.3 Industry and workplace observation 24

2.4.4 Insights from Interviews and Observation 24

2.5 Requirements Analysis **27**

2.5.1 Users and use cases 28

2.5.2 Functional requirements 29

2.5.3 Non-functional requirements 31

2.5.4 Technical requirements 32

2.5.5 Constraints 33

2.6 Conclusion **34**

Chapter 3:Architecture and Design 35

3.1 Introduction **35**

3.2 System Overview and High-Level Architecture **37**

3.2.1 System overview 37

3.2.2 System architecture diagram 38

3.2.3 Mapping of architecture design to requirements 38

3.3 Presentation Layer and Web Portal Design **40**

3.4 PHP Proxy, Session Management, and Authentication Design **42**

3.4.1 Authentication and authorization sequence 44

3.5 FastAPI Job-Management Layer and Data Validation Design **44**

3.5.1 Upload preview, mapping validation, and job creation sequence 46

3.6 Analytics Pipeline and Segmentation Design **46**

3.6.1 Data preprocessing and feature engineering 47

3.6.2 Cluster-count selection and clustering-model comparison 47

3.6.3 Customer-group naming and recommendation design 48

3.6.4 Explainability, chart generation, and report artifacts 48

3.6.5 Pipeline design diagram 49

3.7 Data Persistence, Storage, and Database Design **50**

3.7.1 Entity–Relationship Diagram 52

3.8 Sequence Diagram: Upload Preview, Mapping, Validation, and Processing **53**

3.9 Sequence Diagram: Post-Analysis Dashboard and Segment Narrative Flow **54**

3.10 Sequence Diagram: PDF and Segmented Customer CSV Download **55**

3.11 Conclusion **56**

Chapter 4: Implementation 57

4.1 Introduction **57**

4.1.1 Implementation Philosophy and Justification of Major Choices 57

4.1.1 User Workflows 59

Workflow A: SME business owner using the web application 59

4.2 Tools and Technologies **61**

4.2.1 Technologies Considered but Not Retained in the Final System 64

4.3 Implementation Techniques **65**

4.3.1 Model and algorithm development 65

4.3.2 Backend API and application implementation 68

4.3.3 Database design 70

4.3.4 Web Application Implementation Details 71

4.3.5 AI narrative and segment-profiling implementation 73

4.4 Conclusion **73**

Chapter 5: Testing and Results **75**

5.1 Introduction **75**

5.2 Component Testing **75**

5.3 System-Level Testing **78**

5.4 User Acceptance Testing **88**

5.5 Results and Analysis **89**

5.5.1 Issues identified and resolved during testing 90

5.6 Summary **91**

Chapter 6: Conclusions and Recommendations **92**

6.1 Project Summary **92**

6.2 Assessment of the Project Objectives **92**

6.3 Recommendations **93**

6.4 Limitations and Challenges **94**

6.5 Future Work **95**

6.6 Conclusion **96**

APPENDIX **102**

Appendix 1 : AI Use Declaration **102**

Appendix 2 : AI Tools and Usage **103**

Appendix 3: Coding Architecture **105**

3.1: System Code Architecture 105

Appendix 4: Project Management Gantt Chart **110**

Appendix 5 : Customer360 Semi-Structured Interview Guide : SME Stakeholders **111**

List of Figures

Figure 2.1: Main Users of the Customer360 System

Figure 3.1: Architecture of the Current Customer360 System

Figure 3.2: Authentication and Authorization Sequence

Figure 3.3: Upload Preview, Mapping Validation, and Job Creation Sequence

Figure 3.4:Pipeline Design Diagram

Figure 3.5: Entity–Relationship Diagram

Figure 3.6: Upload Preview, Mapping, Validation, and Processing Sequence Diagram

Figure 3.7: Sequence Diagram for the Post-Analysis Dashboard and Segment

Figure 3.8: Sequence Diagram for PDF and Segmented Customer CSV Download

Figure 4.1: Final Web Workflow Implementation

Figure 4.2: Implemented Analysis Pipeline

Figure 4.3: Implemented Request Chain

Figure 5.1:Terminal Output Showing Successful Execution of the Customer360 Backend Test

Figure 5.2,5.3: Public landing page of the Customer360 system.

Figure 5.4: Sign-in page for authenticated access.

Figure 5.5: Upload page for customer transaction data.

Figure 5.6,5.7: Upload page for customer transaction data.

Figure 5.8: Processing page showing live job progress.

Figure 5.9,5.10,5.11,5.12 : Analytics dashboard showing completed segmentation results.

Figure 5.13: Reports page showing historical analysis runs.

List of Tables

Table 2.1: Rationale for the Iterative Development Approach

Table 2.2: Summary of Themes Emerging from Stakeholder Interviews

Table 2.3: Illustrative Anonymised Interview Excerpts by Theme

Table 2.4: Summary of Observations and Their Requirement Implications

Table 2.5: Functional Requirements of Customer360

Table 2.6: Non-Functional Requirements

Table 2.7: Technical Requirements

Table 3.1: Design Principles of the Customer360 System

Table 3.2: Architecture-to-requirements mapping

Table 3.3: Main Web Portal Pages and Their Design Roles

Table 3.4: PHP proxy scripts and FastAPI routes

Table 3.5: Core job lifecycle states and UI behavior

Table 3.6: Analytics Modules and Their Functions

Table 4.1: Final Implementation Stack of the Deployed Customer360 System

Table 4.2: Tools and Technologies Used Across System Layers

Table 4.2 A: Justification of the main implementation choices

Table 4.3: Technologies Retained, Narrowed, or Excluded in the Final System

Table 4.4: Implemented analytical stages

Table 4.5: Implemented evaluation metrics

Table 4.6: Main Implementation Modules and Their Function

Table 4.7: Current database entities in the deployed schema

Table 4.8: Main Web Pages Implemented in the Current System

Table 5.1: Summary of Automated Component Test Results

Table 5.2: Component testing coverage

Table 5.3: System-level workflow results

Table 5.4: Deployment verification results

Table 5.5: User Acceptance Testing Outcomes

Table 5.6: User Acceptance Testing Outcomes

Table 5.7: Important issues discovered and resolved

Table 6.1: Summary of Project Objectives Achieved

Chapter 1: Introduction

1.1 Background

Small and medium-sized enterprises (SMEs) have been recognized as the base of economic development in developed and developing economies. They make a significant contribution to employment creation, income generation, and overall national productivity. In fact, the OECD [2] noted in 1997 that globalization is changing the competitive environment faced by enterprises of all sizes, while the adjustment burden may be higher for small enterprises in less well-developed countries. In Ghana, those costs remain high. This worldwide pressure is especially strong for SMEs in less developed nations since their capacity to compete is restricted by their lack of access to finance, technology, and trained labor [3].

In Ghana, SMEs play an especially significant role in the national economy. The Ghana Statistical Service (GSS) classifies enterprises based on their number of employees: Businesses that employ one less than 6 people are categorized as micro, 10 to 29 as small, and 30 to 99 as medium[5].According to the Ghana SME Sector Report 2023, over 90% of registered business enterprises in the country are SMEs, forming approximately 80% of total employment while also contributing roughly 60% of the national GDP [4]. Data from the Registrar General's Department further indicates that 92% of companies registered in Ghana fall within the MSME category [4].

Despite this, most Ghanaian SMEs struggle with basic technology adoption. Agboh [6] studied technology adoption among SMEs in Accra and found that limited digital literacy, financial constraints and high costs were the primary barriers. Danquah [7] discovered that although artificial intelligence has the potential to enhance strategic decision-making,

operational efficiency, and customer service among Ghanaian SMEs, its adoption remains limited due to constraints such as inadequate infrastructure, funding challenges, a shortage of skilled personnel, low awareness, and unclear regulatory frameworks. This issue increases by the fact that many SMEs still rely on manual record-keeping, intuition, or simple spreadsheet reports to understand their customers and inform their marketing decisions. They may be aware that some products sell well at particular times, but they lack an organized technique to determine which clients are worth keeping, which are leaving, or which ones react better to discounts than others who always pay full price.

Customer segmentation is the process of categorizing a customer base into distinct groups based on similar characteristics, behaviors, or needs, allowing businesses to engage with each group in more targeted and effective ways [8].When segmentation is used effectively, companies can uncover high-value clients, identify at-risk groups before they leave, and more effectively use marketing resources.

When traditional human methods are unable to handle the volume or complexity of available data, machine learning emerged as a powerful driver of consumer segmentation. Without the use of pre-labeled examples, clustering algorithms such as K-Means, Gaussian Mixture Models, and Hierarchical Clustering can identify groupings in data that a human analyst might overlook. . Clustering becomes useful and comprehensible when paired with the RFM (Recency, Frequency, Monetary) model, which simplifies each client to three integers that represent how recently, how often, and how much they spent [11][12].

However, just a small percentage of Ghanaian SMEs are using machine learning for customer analytics. Many companies lack access to analytical tools capable of processing transaction data and generating useful insights, and those that do often lack the technical know-

how to assess the findings. Badghish and Soomro [13] found that the adoption of artificial intelligence can significantly improve SME performance, particularly in terms of economic and operational outcomes, by enhancing forecasting, reducing costs, automating tasks, and supporting innovation. This project responds to that gap by developing a practical, accessible platform that automates customer segmentation using machine learning and presents the results in a form that non-technical business owners can understand and act on. The primary analysis uses real transaction data obtained from ThreeTwentyOne, a fashion and streetwear brand based in Accra, comprising over 1,500 transactions recorded in Ghanaian Cedis (GHS). This grounds the study in actual Ghanaian businesses data rather than synthetic or international datasets.

1.2 Problem Statement and Justification

Despite Ghanaian SMEs' increasing awareness of the need for digital transformation, their marketing strategies frequently lack the analytical depth required to comprehend the diversity and behavior of their customer. Every customer on the contact list may receive a WhatsApp broadcast about a new collection from a fashion business, regardless of whether the customer is a regular customer who spends GHS 1,000 per visit or someone who purchased one item six months ago and never came back. Broad, intuition-based marketing strategies that treat every consumer equally are currently standard practice for the majority of SMEs, which results in wasteful spending, low customer loyalty, and missed opportunities to hold onto valuable customer segments.

The desired state is one in which SMEs can identify distinct customer groups based on purchasing behavior and tailor their marketing strategies to each group's characteristics and needs. The frequent high-spenders get early access to new drops. The price-sensitive group gets

a targeted discount. The customers who have not bought in months get a re-engagement offer. This kind of targeting is standard practice for large companies with analytics departments, but it remains out of reach for most Ghanaian SMEs.

Many SMEs already collect transaction records through POS systems, mobile money logs, or even manual ledgers. The main limitation is not the availability of data, but the lack of suitable tools to effectively utilize it. Existing segmentation software assumes a level of technical skill that most SME owners do not have, and the cost of hiring someone who does is often prohibitive. ML-driven segmentation research has concentrated on retail, banking, and tourism [14]. Its application to small firms in developing countries like Ghana remains largely unexplored. There is a clear need for a lightweight, affordable, and accessible segmentation tool that is specifically designed for the operational realities of Ghanaian SMEs.

1.3 Aims of the Project

The aim of this project is to design, build, and test a machine learning-based customer segmentation platform for Ghanaian SMEs that can produce actionable customer groups from raw transaction data without requiring technical expertise from the user.

The specific objectives of this project are to:

1.3.1 OBJ 1: Data Collection and Preparation

Identify and analyze customer-related transaction data relevant for segmentation within the Ghanaian SME context, including the development of a flexible data ingestion pipeline that can accept transaction files from different business systems and automatically validate, clean, and preprocess the records for analysis. The primary dataset used in this study was obtained from ThreeTwentyOne, a fashion retail SME based in Accra, and contains over 1,500

transaction records with fields including order identifiers, customer identifiers, purchase dates, product details, quantities, unit prices, and total line amounts.

1.3.2 OBJ 2: Model Development and Deployment

Apply multiple machine learning clustering algorithms, specifically K-Means, Gaussian Mixture Model, and Hierarchical Clustering, to categorise customers into distinct and meaningful segments based on RFM (Recency, Frequency, Monetary) behavioural attributes, and implement an automated pipeline that determines the optimal number of clusters and selects the best-performing algorithm using Silhouette Score.

1.3.3 OBJ 3: Evaluation and Validation

Evaluate the performance and effectiveness of the clustering results using established metrics including Silhouette Score, Calinski–Harabasz Index, and Davies–Bouldin Index, and validate the stability and statistical significance of the resulting segments through repeated-run analysis and ANOVA testing.

1.3.4 OBJ 4: Explainability

Apply Explainable Artificial Intelligence (XAI) methods, specifically SHAP-based feature interpretation through a surrogate Random Forest model, to explain which customer-behavior features most influenced the final customer grouping. SHAP was selected because it provides global feature importance insights that show how Recency, Frequency, and Monetary value each contribute to the overall segmentation, making the results interpretable for SME owners who need to understand why their customers were grouped in a particular way [15].

1.3.5 OBJ 5: System Integration and Recommendations

Deliver the segmentation capability through two integrated platforms: a web application built on FastAPI, PHP, and MySQL for non-technical SME users, and a Google Colab notebook

for researchers and analysts. Each identified customer segment is paired with actionable marketing recommendations to support practical business decision-making.

1.4 Proposed Solution Approach

Customer360 is built around four modules that handle different parts of the pipeline.

1.4.1 Component A: Data Ingestion and Validation Module.

This component provides the guided entry point through which SME users submit transaction data for analysis. The user uploads a transaction file, reviews a preview of the detected columns, checks suggested mappings, and assigns the required business fields before processing begins. To handle variation in SME datasets, the module supports automatic encoding handling, case-insensitive recognition of likely column names, and flexible monetary configuration, where total amount may be read directly from a source column or derived from quantity and unit price. It also lets the user define parsing rules such as date format, decimal separator, thousands separator, currency symbol, and how negative amounts should be treated. To protect system resources and reduce failures, a 25 MB upload limit is enforced at the PHP web layer, and validation is completed before a background job is created. A job is one complete analysis run in the system. It includes the uploaded file being analyzed, the mapping and clustering settings chosen for that run, the current processing state such as pending, processing, completed, failed, or cancelled, the progress information shown on the processing page, the saved outputs produced from the run such as the results JSON, charts, segmented customer CSV, and PDF report, which is connected to an ownership link that ensures only the user who created the job can reopen or download its results.

1.4.2 Component B: Data Processing and Segmentation Engine.

This component performs the core analytical work after the uploaded dataset has passed preview and mapping validation. The pipeline removes fully blank rows, applies the selected field mappings, parses dates and numeric values using the user's locale-aware rules, and derives amount from quantity and unit price where a direct amount field is unavailable. Records that cannot support valid analysis are excluded, after which the cleaned transactions are aggregated to the customer level and converted into Recency, Frequency, and Monetary (RFM) features. To prepare the data for clustering, Frequency and Monetary are log-transformed and the RFM variables are standardized using StandardScaler. The system then evaluates candidate cluster counts from 2 to 8 using internal validation metrics and selects the most defensible value through a majority-vote strategy. K-Means, Gaussian Mixture Model, and Hierarchical Clustering are then compared, and the strongest-performing method is retained for the final result. Each customer segment is assigned a plain-language business label such as 'Your Star Customers', 'Almost Regulars', 'Danger Zone', 'Sleeping Customers', or 'Gone Customers', together with practical marketing recommendations. Where two clusters share the same broad segment meaning, suffixes such as Group 1 and Group 2 are added for clarity.

1.4.3 Component C: Explainability and Visualization Module.

This component makes the segmentation results interpretable to non-technical users. A Random Forest surrogate model is trained on the scaled RFM features using the final cluster assignments as target labels, and SHAP is applied to estimate which behavioral dimensions most strongly influenced the customer groups. The system also generates analytical visuals including segment-size charts, radar charts, PCA-based customer maps, buying-pattern

distributions, and feature-importance summaries. In the deployed web application, these outputs are presented mainly as generated chart images delivered through the PHP layer rather than as a fully interactive charting environment. The module further supports usability through plain-language KPI explanations, readable feature descriptions, and optional segment-level AI profile actions, helping SME users understand both how customers were grouped and what those groupings mean in practical business terms.

1.4.4 Component D: Automated Reporting Module.

This component generates a downloadable PDF report for each completed analysis. In the current implementation, the report is produced with ReportLab and includes a title page, executive summary, data overview, RFM analysis, segmentation results, visual analysis, detailed segment profiles, and business recommendations. Its purpose is to give SME users a portable and structured summary of the analysis that can be revisited after the session or shared with stakeholders for planning and decision-making. Rather than serving as a highly technical appendix, the report is designed to communicate the most important findings in clear business language, supported by generated visuals and recommended actions linked to the identified customer groups.

1.5 Relevance of the Project

This project addresses a practical need within Ghana's SME sector by translating academic research on machine learning-driven segmentation into an accessible tool that business owners can use without programming knowledge. While existing research consistently demonstrates the benefits of customer segmentation for business performance [9][16], few

studies have attempted to deliver these capabilities through a platform designed specifically for resource-constrained SMEs in developing countries.

By demonstrating that machine learning-based analytics can be made affordable and readily available for all small businesses, this project also contributes to Ghana's larger goal for digital transformation in all aspects of its economy. By supporting SME competitiveness, it supports SDG 8 (Decent Work and Economic Growth) and SDG 9 (Industry, Innovation, and Infrastructure) by encouraging technical innovation in the context of developing economies. The system's insights may also help governments and business development organizations create capacity-building initiatives that encourage local businesses to make data-driven decisions.

The primary segmentation analysis was conducted on a real transaction dataset obtained from ThreeTwentyOne, a fashion and streetwear SME based in Accra. The dataset contains over 1,500 transactions with fields including hashed order identifiers, spoofed customer identifiers, purchase dates, order statuses, payment and shipping methods, product names and categories, colors, sizes, quantities, unit prices, and total line amounts, all denominated in Ghanaian Cedis (GHS). This grounds the study in actual Ghanaian business data, providing locally relevant evidence of how clustering-based segmentation can generate actionable customer insights within Ghana’s retail environment.

The project also makes a methodological contribution by integrating explainable AI into the segmentation pipeline. The use of SHAP via a surrogate model addresses the challenge of applying post-hoc explainability to unsupervised clustering, an approach that has received

growing attention in the literature but has rarely been implemented in a tool designed for non-technical end users [15].

1.6 Conceptual Definitions

1.6.1 Customer Segmentation

Customer segmentation is the process of categorizing a company's customers into groups of people who have similar characteristics. The goal is to help organizations treat various groups differently, whether through focused marketing, customized product offerings, or varied approaches to pricing. Tsiptsis and Chorianopoulos [17] identify four broad approaches to segmentation: behavioral, psychographic, profitability, and demographic. Of these, behavioral segmentation based on purchase history tends to be the most actionable for small businesses, because the data it requires (who bought what, when, and for how much) is already captured in transaction records.

1.6.2 The RFM Model

The RFM model reduces each customer's purchase history to three values. Recency determines how many days have passed since the customer's most recent transaction. The frequency measures how many separate transactions the consumer has performed in a given time period. Monetary sums up the total amount the customer has spent. Hughes[18] introduced this approach in the context of direct marketing, and it has become one of the most extensively used feature engineering methodologies for consumer segmentation [11][12]. One of its key advantages is the model's simplicity as it requires only three core data fields (customer ID, transaction date, and transaction amount), making it available to organizations with limited data.

1.6.3 Unsupervised Clustering Algorithms

Clustering is a category of machine learning where the algorithm groups data points based on similarity without being told in advance what the groups should look like. K-Means is the most commonly used clustering algorithm in customer segmentation research [19]. K-Means works by placing K central points(centroids) within the feature space. Each data point is then assigned to the centroid it is closest to. After this, the centroids are updated by calculating the mean of the points assigned to each cluster. This process continues until the cluster assignments no longer change significantly. Gaussian Mixture Models (GMM) follow a different process by assuming that the data is generated from a mixture of Gaussian distributions and then applying the Expectation-Maximization algorithm to estimate the parameters of each distribution [20]. Hierarchical Clustering takes another approach by creating a tree-like structure of clusters through the repeated merging of the closest pair of clusters, using a linkage method such as Ward's method to reduce variation within each cluster [10].

1.6.4 Cluster Evaluation Metrics

Because clustering is unsupervised, there is no ground truth to compare the results against. Instead, internal validation metrics are used. Rousseeuw [21] proposed the Silhouette Score, which measures how similar each data point is to its own cluster against neighbouring clusters on a scale of -1 to 1. The Calinski–Harabasz Index measures the ratio of between-cluster dispersion to within-cluster dispersion. The Davies–Bouldin Index measures the average similarity between each cluster and the cluster most similar to it, where lower values indicate better separation [31]. Different metrics can recommend different values of K for the same dataset, which is why some researchers use a majority-vote approach across multiple metrics.

1.6.5 Explainable AI and SHAP

Explainable AI (XAI) is a set of methods that allows humans to understand outputs of machine learning models. SHAP (SHapley Additive exPlanations), introduced by Lundberg and Lee [15], computes the contribution of each feature to a given prediction using Shapley values from cooperative game theory. The issue with clustering is that SHAP was designed for supervised models. A common approach to this is to train a supervised surrogate model using the cluster labels and then apply SHAP to that model instead. This allows the researcher to see which features the surrogate model relies on most when predicting cluster membership, which can be used as a substitute to better understand what drives segmentation.

1.7 Empirical Issues

1.7.1 Performance of Clustering Algorithms in Customer Segmentation

The most suitable clustering technique for consumer segmentation depends largely on the structure of the data, especially the shape and spread of the clusters. John et al. [20] compared clustering algorithms K-Means, GMM, DBSCAN, Agglomerative Clustering, and BIRCH on a UK retail dataset with over 540,000 transactions using RFM features. GMM achieved the highest Silhouette Score (0.80), while K-Means scored 0.64. But K-Means was faster to run and produced clusters that were easier to interpret. A separate comparative study [24] on the Mall Customer Segmentation dataset found that K-Means, Davies–Bouldin Index, BIRCH, and the hybrid K-Means + DBSCAN model produced more stable results than GMM when the cluster shapes were roughly spherical, but GMM handled elongated or overlapping clusters better.

The practical takeaway is that no single algorithm is universally superior. K-Means works well when clusters are compact and roughly round, which is often the case with RFM data after log transformation and scaling. GMM is more flexible but slower and can overfit. Hierarchical Clustering with Ward linkage is deterministic but struggles with large datasets [24]. This is why Customer360 runs all three and selects the best performer automatically.

1.7.2 Limitations of the RFM Model

RFM is popular because it is simple and interpretable, but it has its limitations. Lewaa et al. [25] pointed out that RFM does not account for customer demographics, product category preferences, or the nature of the items purchased. According to Wang et al. [26], the Recency component is subject to randomness: a consumer who makes a purchase just before the study date appears to be highly involved, even if they are usually infrequent. Several extensions have been proposed, including LRFM (adding Length of relationship), and RFM-A (adding Age) [26][27]. Alves Gomes and Meisen [19] highlight that many customer segmentation approaches rely on common techniques such as RFM, with K-Means being the most widely used method. Although this combination is practical and commonly applied, its simplicity can be a limitation, as it may not fully capture more complex patterns in customer behaviour

1.7.3 XAI Adoption in Customer Analytics

Explainability techniques are becoming more widely used in consumer analytics, although their application remains limited. Maqbool and Qureshi [28] conducted a systematic review of 80 studies published between 2015 and 2025 and discovered that SHAP and LIME are the two most commonly used XAI methods in customer-facing models, but the vast majority of applications are in churn prediction (supervised) rather than segmentation. When SHAP has

been applied to clustering, it has typically been through a surrogate model approach, in which a supervised classifier is trained on cluster labels and SHAP values are then computed from that classifier. Molnar [23] covers SHAP as a way to measure how much each feature contributes to a model's predictions, and also discusses surrogate models, where an interpretable model is trained to approximate a less interpretable one. This method is simple to implement and produces interpretable results.

1.7.4 Data Quality and Analytics Challenges for African SMEs

The challenges facing SMEs in Africa when it comes to data analytics differ from those in developed economies. According to Majeed et al. [29] African SMEs struggle with insufficient funding, complicated data, a lack of analytics tools, and unreliable infrastructure, among other barriers. Butala and Mwanza [30] studied 220 SMEs in Lusaka and found significant gaps in both data analytics knowledge and actual application in supply chain decisions. Across these studies, the issue is less about data availability and more about whether SMEs have the tools, skills, and support to actually use it. An analytical tool built for this audience would need to be cheap, straightforward, and capable of handling the kind of messy, incomplete data that African SMEs usually have.

1.8 Research Gaps

Based on the literature reviewed, four specific gaps can be identified that motivate this project:

Gap 1: Most segmentation research focuses on e-commerce, not SMEs. Alves Gomes and Meisen [19] reviewed 105 customer segmentation studies and found the research concentrated in e-commerce settings, with K-means as the dominant method. None of the reviewed studies examine whether these methods work with the smaller, nosier datasets typical of SMEs in developing countries.

Gap 2: Limited research on ML-driven segmentation in the Ghanaian or African SME context. RFM-based customer segmentation has been widely studied in e-commerce and retail settings, with the bulk of peer-reviewed work using datasets from the UK [11][20] and Southeast Asia [12]. Research applying these methods to African SME transactional data remains scarce, with very little attention given to the Ghanaian context.

Gap 3: No accessible web-based segmentation platform exists for SMEs in developing countries. While Colab notebooks and Python scripts for RFM segmentation are widely available, they require programming knowledge. Commercial tools like HubSpot or Salesforce are priced beyond what most Ghanaian SMEs can afford.

Gap 4: XAI is rarely integrated into segmentation tools aimed at non-technical users. Maqbool and Qureshi [28] found that XAI methods in customer analytics are overwhelmingly applied to churn prediction rather than segmentation. While SHAP has been used to explain clustering results through surrogate models [23], these approaches remain confined to technical workflows and have not been packaged into tools that non-technical users can interact with.

1.9 Conceptual Framework and Comparative Analysis

This section describes the reasoning behind the main technical decisions.

1.9.1 RFM as the Feature Engineering Approach

RFM was chosen because it matches the type of data that Ghanaian SMEs actually have. A fashion retailer with a spreadsheet of orders can supply a customer ID, a date, and an amount. That is enough for R, F, and M. More complex feature sets involving demographic data or browsing behavior would produce richer segments, but they would also require data that most SMEs do not collect [25].

1.9.2 Multi-Algorithm Clustering with Majority-Vote K Selection

A key issue in clustering is selecting the most appropriate number of clusters since different algorithms and validation metrics often produce different results. To address this limitation, some studies have adopted a multi-algorithm clustering framework combined with majority-vote K selection. Ullah et al.[22] applied this approach to a large Pakistani e-commerce dataset comprising 584,524 transactions. Their framework combined four clustering algorithms : K-Means, Agglomerative Clustering, Gaussian Mixture Models, and DBSCAN : with four internal validation metrics: Silhouette Score, Calinski–Harabasz Index, Davies–Bouldin Index, and Dunn Index. Each algorithm–metric combination generated a recommended cluster count across a range of 2 to 10 . Rather than relying on a single algorithm or metric, the final value of K was selected through majority voting, using the statistical mode to identify the most frequently recommended cluster count across all evaluations. This approach is important because different clustering algorithms capture different structural patterns in data and may

therefore suggest different values of K. By combining results across multiple methods, it provides a more balanced and reliable basis for cluster determination.

1.9.3 SHAP Surrogate Model for Explainability

Although clustering is effective for identifying meaningful customer groups, it does not directly explain the feature-level basis of cluster membership. This creates an interpretability gap in applied settings where segmentation results must be clearly justified. To address this, the present study adopts a surrogate-model explainability approach based on SHAP, which assigns each feature an importance value for a given prediction [15]. In this framework, the cluster labels generated during clustering are used as outputs for a supervised model, such as Random Forest, after which SHAP is applied to estimate the contribution of each feature to customer segment membership. Recent work has shown that integrating clustering, classification, and Shapley-value analysis can improve transparency by revealing the variables that most strongly influence how customers are grouped into segments [32].

However, this approach should be interpreted carefully. SHAP explains the predictions of the supervised model used after clustering rather than the internal mechanism of the clustering algorithm itself [15]. A study by Shu et al [33] uses SHAP to interpret the feature contributions associated with the derived clusters. This suggests that SHAP was applied after clustering to explain cluster assignments, rather than to explain the internal mechanism of the K-Means algorithm itself. This reflects how the surrogate model learns and reproduces cluster membership from the input features. Nonetheless, the integration of clustering, classification,

and explainability provides a practical basis for interpreting segment structure within applied customer segmentation context.

1.10 Comparative Analysis of Technologies

Table 2.1: Comparison of Clustering Algorithms

Criterion	**K-Means**	**GMM**	**Hierarchical**	**Customer360**
Cluster shape assumption	Best for compact, roughly spherical clusters	Can model elliptical and more flexible cluster shapes	No explicit shape assumption	Runs all three and selects the best-performing method
Speed	Fast	Moderate	Slow for large datasets	Acceptable for SME-sized datasets
Deterministic?	No, unless initialization is fixed	No, unless initialization is fixed	Yes	Uses fixed seeds for reproducibility where needed
Soft assignment	No (hard only)	Yes, probabilistic assignment	No (hard only)	Uses hard labels for simplicity
Interpretability	High	Moderate	High especially with dendrograms	Improved through SHAP surrogate explanations
Best suited for	Compact, well-separated RFM clusters	Overlapping clusters and more complex distributions	Smaller datasets or nested relationships	Adapts per dataset

1.10.1 Organization of Report

Chapter 2: Literature Review. This chapter reviews existing research on customer segmentation, the RFM model, clustering algorithms and their comparative strengths, the challenges of SME data in developing country contexts, and the application of explainable AI techniques in marketing analytics. It establishes the theoretical and empirical foundation for the design decisions made in subsequent chapters.

Chapter 3: Requirements. This chapter identifies the key stakeholders, describes the requirements gathering process, defines the functional and non-functional requirements, presents the proposed solution, and discusses the ethical considerations involved in handling customer data under Ghana's Data Protection Act (Act 843).

Chapter 4: Architecture and Design. This chapter presents the system architecture, data flow, and design decisions for the Customer360 platform, including the RFM feature engineering pipeline, multi-algorithm clustering approach, segment labelling logic, SHAP explainability module, and PDF report generation.

Chapter 5: Implementation. This chapter explains how the system was built across the FastAPI backend, PHP frontend, MySQL database, and analytical pipeline, including the tools and technologies used, database schema, API implementation, and frontend design.

Chapter 6: Testing and Results. This chapter presents the testing strategy, unit tests, API tests, integration tests, and planned user acceptance tests, alongside the analytical results and a discussion of the system's strengths and limitations.

Chapter 7: Conclusions and Recommendations. This chapter summarizes the extent to which the project met its objectives, discusses the limitations of the current implementation, and provides recommendations for future development.

Chapter 2: Requirements

2.1 Introduction

This chapter presents the requirements analysis for Customer360. It identifies the needs of the intended users and translates them into functional, non-functional, technical, and constraint-based requirements. These requirements serve as the foundation for the architecture and implementation chapters that follow. This phase was important because Customer360 was developed for small and medium-sized enterprises in Ghana, where transaction records are often inconsistent, customer analysis is usually carried out informally, and many users may not have the time or the capacity to work through technical processes.

2.2 System Overview

Customer360 is a web-based platform that helps SMEs analyze transaction data and convert it into interpretable customer groups and practical business recommendations. The system workflow follows a systematic approach where a user signs in, uploads a CSV file, previews the detected columns, maps the required business fields, validates parsing rules, submits an analysis job, monitors progress, and reviews the completed results through an analytics dashboard and downloadable report outputs. At the data-processing level, the backend cleans uploaded records, derives Recency, Frequency, and Monetary features, compares clustering approaches, assigns business-friendly customer-group labels, and generates charts, reports, and segmented customer exports. At the user level, the system is intended to make this process accessible through a guided workflow and plain-language outputs rather than requiring programming knowledge.

2.3 Project Management Methodology

The project was developed using an iterative and incremental approach, which made it suitable for the system design made up of several connected components, including the web interface, analytical backend, database integration, reporting, and optional AI-assisted summaries, which could not be fully refined in one iteration. Instead, the system developed gradually, with the workflow, deployment structure, and interface language being improved over time as real datasets were tested and practical issues became clearer.

The project involved several connected components, including the PHP pages, FastAPI routes, MySQL storage, file outputs, clustering logic, and report generation. The data-ingestion process also had to be adjusted to reflect the nature of typical SME datasets. Initial assumptions that users could move directly from file upload to analysis proved unrealistic, and with that, a preview and mapping-validation feature was introduced after different file structures were observed. The deployment model evolved as the project developed. The final setup design colocated Apache, PHP, and FastAPI on the same host, and was adopted because it proved more stable in practice than the earlier arrangement. The language used in the dashboard and reports also had to be refined so that the results would be easier for non-technical SME users to understand.

Table 2.1: Rationale for the Iterative Development Approach

Project Condition	**Relevance to the Iterative Approach**
Diverse SME datasets	Mapping and parsing rules had to be refined after testing with real files
Hybrid architecture	PHP, FastAPI, MySQL, reporting, and AI layers had to be integrated gradually
Live deployment changes	Backend hosting and routing decisions evolved during implementation

User-facing wording	Segment names, card labels, and PDF wording needed revision after review
Reliability fixes	Report, serialization, and export issues were identified and corrected incrementally

2.4 Requirement Gathering

Requirements were gathered through three methods including review of literature and related systems, semi-structured stakeholder interviews, and direct observation of SME business practice. The three methods were chosen together because the problem is both technical and contextual. The technical side covers customer segmentation methods, RFM analysis, clustering evaluation, explainability, and web system design. The contextual side covers how Ghanaian SMEs keep records, how customer decisions are currently made, and what kinds of outputs would be useful in practice.

2.4.1 Literature and related-system review

The literature review identified segmentation approaches, strengths and limitations of the RFM framework, suitable clustering algorithms, validation metrics, explainability techniques, and technology-adoption problems affecting SMEs. This provided the analytical foundation for the project, but it could not fully capture how SMEs organize their records in practice or what type of interface they would find easy to use.

2.4.2 Stakeholder interviews

Semi-structured interviews were conducted with SME owners, managers, and individuals involved in SME operations or business roles. These interviews helped identify the everyday difficulties businesses face in tracking customers, making marketing decisions, and managing records. The interviews also revealed the kinds of outputs users preferred, how much workflow complexity they could handle, and which insights they found most useful. Insights

gathered from interviews showed that customer information is often available but not in a clear or organized form. Records are usually kept in WhatsApp chats, Google Sheets, spreadsheets, or traditional notebooks. Frequent customers are typically recognized by phone number, email address, or familiarity rather than through any formal analysis. Participants expressed interest in knowing which customers spend the most, which ones are likely to return, which are becoming inactive, and how to better direct promotions and effective marketing strategies toward the right customers.

2.4.3 Industry and workplace observation

Observation was used to understand how businesses manage customers and order information in practice, outside formal interview responses. These observations showed that record-keeping is often manual or semi-manual. Most businesses know what sells but do not analyze customer behaviour beyond that. Decisions are made from memory, intuition, handwritten notes, or recent sales. Observations also showed that output language needs to be understandable and non-technical.

2.4.4 Insights from Interviews and Observation

Table 2.2: Summary of Themes Emerging from Stakeholder Interviews

Participant Type	Main Findings from the Interview	Requirement Implication
E-commerce or web-operations participant in fashion retail	Orders and payments were mainly received through the website, WhatsApp, and pop-up events. Repeat customers were usually identified through phone numbers or email addresses. High-spending customers sometimes received discount offers. The participant wanted segmentation to support	Need for customer grouping, identification of high-value customers, repeat-customer tracking, and actionable campaign recommendations

	more targeted advertising and a clearer understanding of customers.	
Small apparel business owner	Customer records were kept inconsistently, sometimes in message histories or informal notes. Frequent customers were recognized mainly from memory or phone numbers. The participant described marketing and customer conversion as major challenges and wanted to know which customers were worth targeting more intentionally.	Need for an accessible upload process, plain-language summaries, and practical marketing-focused outputs
Custom-fashion or tailoring participant	Customer management was highly personal, with clients informally grouped by spending power, responsiveness, or ease of service rather than through formal analysis. The participant felt segmentation could still help in deciding where to focus marketing efforts and which customers to prioritize.	Need for business-friendly grouping logic and recommendations that connect directly to customer value and follow-up effort
Food-delivery or order-management participant	Orders were captured through WhatsApp and managed through a mix of Google Sheets and shared message threads. Repeat customers were recognized by phone number. One desired insight was the identification of geographic hotspots to support branch or delivery decisions.	Need for flexible CSV imports, customer-matching fields, and outputs that can highlight location or demographic patterns where data is available

Table 2.3: Illustrative Anonymized Interview Excerpts by Theme

Theme	**Paraphrased Excerpt**
Informal repeat-customer tracking	"We usually identify repeat customers through their phone numbers or from memory rather than through a structured system."
Inconsistent record-keeping	"Some records are kept in Excel or Google Sheets, but much of the customer history still remains in chats or manual notes."
Need for targeted marketing	"If we knew which customers spend more or are likely to return, we could target promotions more effectively instead of sending the same message to everyone."
Need for practical outputs	"A PDF summary or clear explanation would be useful, as long as it is easy to understand and act on."
Need for customer insight beyond total sales	"We know which products are selling, but we do not always know enough about the kinds of customers buying them."

Table 2.4: Summary of Observations and Their Requirement Implications

Observation Context	**Observation**	**Implication**
SME offices and shops	Record-keeping was often manual or partly manual, with some businesses using books, message histories, basic spreadsheets, or ad hoc Google Sheets.	The system needed guided upload, flexible column mapping, and validation that could handle imperfect data.
Customer follow-up practice	Customer decisions were often based on memory, intuition, or recent interactions rather than formal analysis.	The system needed to show reusable customer groups and longer-term insights, not just one-time summaries.

Product and demand tracking	Businesses often knew which products or periods were active, but did not usually carry out customer-level analysis to explain why.	The system needed to link transaction records to customer groups and practical business recommendations.
Everyday working conditions	Users needed simple language and direct outputs rather than technical statistical terms.	The interface and reports needed plain-language explanations and practical recommendations.

2.5 Requirements Analysis

The requirements identified through these methods are organized into four main categories: functional, non-functional, technical, and constraint-based requirements. This structure makes it possible to differentiate between the system's expected functions, its performance and quality expectations, the technologies needed for implementation, and the limitations within which the project had to be developed. Taken together, these requirements form the basis for the design decisions, implementation choices, and later verification of the Customer360 system.

2.5.1 Users and use cases

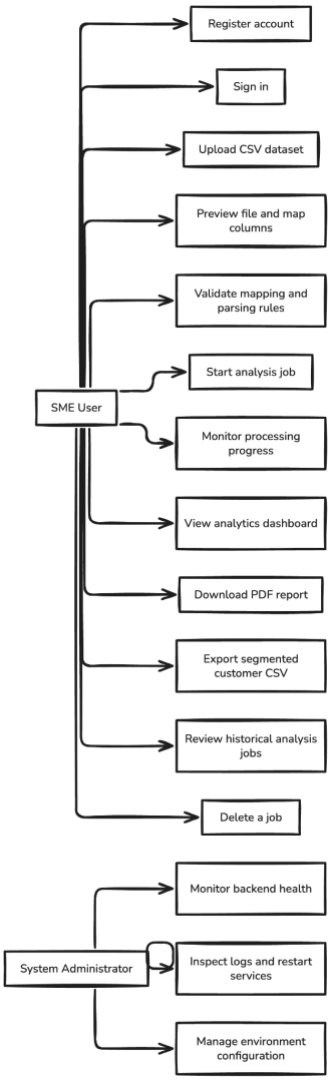


Figure 2.1: Main Users of the Customer360 System[1]

[1] For a higher-resolution version of this figure, click here

2.5.2 Functional requirements

Table 2.5: Functional Requirements of Customer360

Requirement ID	Functional Requirement	Linked Objective(s)	Source
FR001	The system shall allow authenticated users to upload a CSV transaction dataset for analysis.	OBJ 1, OBJ 5	Interviews, observation
FR002	The system shall preview uploaded CSV files and display detected columns before analysis begins.	OBJ 1	Observation, implementation refinement
FR003	The system shall allow the user to map source columns to required business fields such as customer ID, invoice date, invoice ID, amount, quantity, and unit price.	OBJ 1	Observation, interviews
FR004	The system shall validate parsing rules, including date format and numeric separators, before creating a background job.	OBJ 1	Observation, implementation refinement
FR005	The system shall clean and preprocess transaction records, including blank-row removal, invalid-row filtering, and derived amount calculation where needed.	OBJ 1, OBJ 2	Literature review, implementation refinement
FR006	The system shall compute customer-level RFM features from the cleaned transactional data.	OBJ 2	Literature review

FR007	The system shall evaluate multiple clustering methods and select the best-performing method for the job.	OBJ 2, OBJ 3	Literature review
FR008	The system shall assign readable, business-facing customer group labels and segment descriptions.	OBJ 2, OBJ 5	Interviews, observation
FR009	The system shall generate charts, dashboard summaries, and downloadable PDF and CSV outputs for completed jobs.	OBJ 4, OBJ 5	Interviews, observation
FR010	The system shall display live job progress and completion status to the authenticated user.	OBJ 5	Interviews, implementation refinement
FR011	The system shall store completed jobs so that the authenticated owner can revisit past analytics results.	OBJ 5	Interviews
FR012	The system shall restrict access so that users can only view or download jobs they own.	OBJ 5	Security best practice, implementation needs
FR013	The system shall provide an optional plain-language AI-assisted narrative without exposing raw customer identifiers externally.	OBJ 4, OBJ 5	Implementation refinement

2.5.3 Non-functional requirements

Table 2.6: Non-Functional Requirements

Requirement ID	Non-Functional Requirement	How It Was Checked
NFR001	The guided upload, mapping, and job-submission workflow must work through a standard desktop web browser without requiring users to install additional software.	Browser-based workflow test
NFR002	Under normal operating conditions, the full analytical pipeline must complete within 60 seconds for a CSV file containing up to 10,000 transaction rows in the target deployment environment.	Timed benchmark on the deployed server
NFR003	Upload-preview and mapping-validation responses must be returned within 10 seconds for typical SME CSV files under normal operating conditions.	Timed API or browser workflow test
NFR004	Submitted jobs must display a progress state so that users can tell whether analysis is pending, processing, completed, or failed.	Workflow and interface verification
NFR005	Protected dashboards, reports, and exports must be accessible only to authenticated users.	Authentication and ownership testing
NFR006	In the deployed architecture, the analytical backend must remain private to the server, with browser-facing access passing through the PHP layer.	Deployment inspection and route validation
NFR007	Uploaded files and generated outputs must be kept in job-specific storage so that users can revisit completed	File-path and retrieval verification

	analyses until the job is deleted or a retention policy is introduced.	
NFR008	During supervised test sessions, the deployed system must remain responsive across sign-in, upload, mapping, processing, and result retrieval.	Deployment regression checks
NFR009	Customer groups and recommendations must be presented in plain business language suitable for non-technical SME users.	Interface and report review

2.5.4 Technical requirements

Table 2.7: Technical Requirements

Requirement ID	**Technical Requirement**	**Rationale**
TR001	The public web interface must run through Apache and PHP on the deployment server.	Supports the implemented page-driven workflow and session handling
TR002	The analytical backend must run in Python using FastAPI and Uvicorn.	Supports preprocessing, clustering, explainability, and reporting
TR003	The system must use a relational database to store users, jobs, progress, and related metadata.	Matches the current MySQL-backed implementation
TR004	The system must support CSV input as the main guided upload format.	Aligns with the current mapped web workflow
TR005	The analytical pipeline must use Python data-processing and machine-learning libraries	Supports the current implementation and project objectives

	suitable for RFM segmentation and internal metric evaluation.	
TR006	The system must generate downloadable PDF and CSV outputs for completed jobs.	Supports reporting and reuse of results

2.5.5 Constraints

The development of Customer360 was shaped by a number of practical constraints.

- Budget constraint: The project had to remain affordable and realistic for prototype-level hosting. This influenced the decision to use a colocated server design and to keep the AI narrative feature optional.
- Data-quality constraint: SME datasets were expected to be incomplete, inconsistent, or manually maintained. This made it necessary to include upload preview, column mapping, validation, and more tolerant preprocessing steps.
- User-skill constraint: Many intended users were likely to be non-technical and unfamiliar with analytics terminology. As a result, the system needed a guided workflow and outputs written in plain business language.
- Deployment constraint: The analytical backend could not rely on unstable laptop-hosted services or changing public IP addresses. This encouraged the use of a localhost FastAPI service behind the Apache and PHP layer.
- Time and scope constraint: It was not possible to support every business-data format or enterprise-level feature within a single project cycle. This required tighter scope control around a CSV-first workflow and a prototype-level set of operational features.

2.6 Conclusion

This chapter defines the requirements for Customer360 based on stakeholder needs, observed SME practice, literature-based analytical decisions, measurable quality targets, technical constraints, and project limitations. The requirements are supported by interview and observation evidence and avoid unverifiable claims, particularly around system availability. The requirements describe the workflow that the final system implements: CSV upload, preview, field mapping, parser validation, background processing, analytics review, and report or export retrieval. These requirements form the foundation for the architecture and design presented in the next chapter.

Chapter 3:Architecture and Design

3.1 Introduction

This chapter describes the architecture and design of the Customer360 platform. Customer360 is a browser-based customer segmentation application built to help small and medium-sized enterprises transform transaction records into plain-language customer groups, marketing recommendations, visual charts, and downloadable reports without requiring the business owner to write code or understand machine learning implementation details.

The design of the system is guided by six implementation principles:

- **Modularity** : The system is split into separate components for authentication, file upload, column mapping, analytics processing, report generation, and report history. Each one handles a single job. A bug in the column mapping logic, for instance, can be fixed without touching the report generator or the login flow.
- **Accessibility** : Most SME owners are not data scientists and for this reason, the interface uses plain business language in its results and walks users through CSV preview and column mapping one step at a time, rather than expecting familiarity with RFM terminology from the start.
- **Security** : FastAPI runs on localhost behind Apache, so it never faces the internet directly. Authentication goes through PHP sessions with JWT forwarding, which means only logged-in users can trigger analysis on their customer data. Each user can only see their own result, blocked access to someone else's data. On the frontend, raw server file paths are kept out of responses to avoid giving away internal directory structures.

- **Transparency and Interpretability** : Technical cluster labels like "Cluster 0" are replaced with readable segment names such as ‘Star Customers’, helping business owners immediately understand and act on their generated segments, without needing to know what RFM or clustering means. KPI cards include tooltip explanations so users understand what each number actually represents. The SHAP-based feature importance shows factors driving segmentation, and the PDF report explains findings in business terms rather than leaving users to interpret raw values.
- **Reproducibility** :Each analysis run is assigned a unique job ID and stored in a dedicated output directory. All generated files, including charts, CSV exports, and PDF reports, are preserved separately per each run. Running a new analysis does not overwrite previous results, so past outputs remain accessible.
- **Extensibility** : The analytics pipeline is written as modular Python scripts, with results served through structured API routes. This architecture makes it straightforward to add new features such as additional reporting formats, or new visualization types without needing to redesign the frontend or restructure the existing codebase.

Table 3.1: Design Principles of the Customer360 System

Design Principle	**Implementation**	**Purpose**
Modularity	Separate PHP pages, PHP API proxy scripts, FastAPI routes, analytics modules, and report generation modules	Makes the system easier to maintain and extend without breaking unrelated parts
Accessibility	Guided upload, column mapping, validation feedback, plain-language segment names, KPI help popups, and PDF summaries	Supports SME users with different levels of technical experience

Security	Session checks in PHP, JWT forwarding to FastAPI, per-user job authorization, localhost-only FastAPI service, and safe filename display	Reduces unauthorized data access and avoids exposing server internals
Transparency	Segment explanations, SHAP feature summaries, data-quality panels, and plain-language Groq business narratives	Helps users understand why each group exists and what action to take
Reproducibility	One UUID job_id per run, per-job upload/output folders, and stored result JSON	Makes previous analyses traceable and reloadable
Extensibility	Python analytics pipeline, JSON result contracts, and reusable chart/report endpoints	Allows future improvements without changing the full web application stack

3.2 System Overview and High-Level Architecture

3.2.1 System overview

Customer360 works as a hybrid web application. Apache serves the PHP pages that users see and interact with. On the same Ubuntu server, a FastAPI service handles the actual work: customer segmentation, chart generation, AI narration, and report building. The FastAPI service is not publicly accessible; PHP handles all communication between the user's browser and FastAPI. MySQL keeps track of user accounts and analyzes job records. Uploaded datasets and all generated output charts, CSV exports, PDF reports are stored in dedicated folders on the server filesystem.

At the high level, a user signs in through the PHP frontend and uploads a CSV file. The file is first sent to the backend for preview and column profiling, then the user maps the relevant columns and validation rules on the column mapping page. After validation passes, a

background analysis job is created in FastAPI. The Python pipeline preprocesses the file, computes customer-level RFM features, compares candidate clustering models, assigns business segment labels, generates charts, creates customer-level output rows, and prepares a PDF report. The backend also sends aggregate segment summaries to Groq and stores plain-language narrative responses per job. The processing page polls job progress until completion, after which the analytics page reads the saved results and presents the dashboard, charts, segment cards, AI profile buttons, and export options.

3.2.2 System architecture diagram

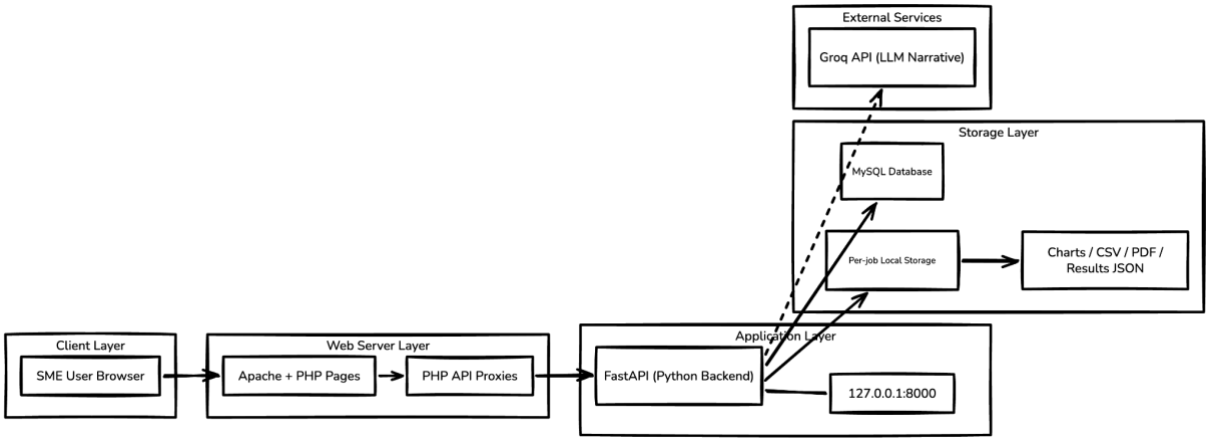


Figure 3.1: Architecture of the Current Customer360 System[2]

3.2.3 Mapping of architecture design to requirements

The architecture is shaped directly by the requirements outlined in the previous chapter. Each layer in the system exists to address a specific set of those requirements. The table below maps design components to their corresponding requirements, showing how the implementation decisions are related to requirements analysis.

[2] For a higher-resolution version of this figure, click here

Table 3.2: Architecture-to-requirements mapping

Architecture Component	**Supported Requirement**	**Design Rationale**
PHP authentication pages + PHP session handling	User registration, login, authenticated access, understandable feedback, and role-restricted access	Keeps browser pages simple while enforcing page-level session checks before users can access upload, analytics, and reports
PHP API proxy scripts	Secure API communication, protected token forwarding, clean browser integration, and controlled error handling	The browser talks to same-origin PHP endpoints, and PHP forwards JWT-authenticated requests to FastAPI privately
FastAPI auth routes	Account creation, login validation, and current-user lookup	Centralizes credential validation and token issuance in the backend
Upload preview + mapping validation flow	CSV upload, flexible column mapping, parser configuration, and early validation feedback	Reduces failed jobs by allowing users to confirm fields and formatting rules before processing begins
Background job processing + progress telemetry	Asynchronous analysis, progress visibility, clear status messages, and practical response time	Long-running analysis does not block the page; the processing screen polls status, progress_percent, progress_stage, and progress_message
RFM + clustering pipeline	Customer segmentation, model comparison, and evaluation metrics	Converts raw transactions into customer-level features and chooses a segmentation model using the implemented validation logic
SHAP summary + plain-language labels	Interpretability and non-technical explanation of group meaning	Explains which customer behavior dimensions influenced grouping and avoids exposing raw model jargon as the primary user-facing language

Groq narrative layer	Automated business summary and segment-level recommendations	Generates one extra plain-language summary per completed job using aggregate-only data, while the main analysis still works if Groq is unavailable
ReportLab PDF generation + CSV export	Downloadable reports and reusable business outputs	Produces a PDF report with tables, visuals, and recommendations, and exports a cleaned customer CSV
MySQL + local job folders	Persistent user/job tracking, historical reports, and reproducible output storage	Job metadata and progress are stored in MySQL, while uploaded files, result JSON, charts, and reports are stored in per-job server folders

3.3 Presentation Layer and Web Portal Design

The presentation layer consists of a set of PHP-rendered pages styled with Tailwind CSS and Material Symbols. This layer is responsible for account onboarding, authenticated navigation, file upload, column mapping, job progress display, dashboard presentation, report history, and user help.

The landing page introduces Customer360 and directs users to registration or login. The registration and sign-in pages collect credentials and business identity information, then the authenticated pages take over once a valid session is established. The dashboard provides recent job visibility and quick navigation. The upload page allows drag-and-drop or manual file selection and now links to a mapping-compatible RFM CSV template whose headers match the expected guided mapping flow. The column mapping page asks the user to choose the file columns for customer ID, date, invoice ID, amount, quantity, unit price, product, and category, and it also lets the user configure date format, decimal separators, thousands separators,

currency symbols, negative amount policy, and whether synthetic fallback fields should be allowed.

The processing page shows backend progress by polling the job status endpoint. The analytics page presents KPI cards, a Groq business summary panel, a feature-importance section, chart tabs, segment cards, on-demand AI profile buttons, a customer explorer table, and a data-quality summary. The reports page shows historical jobs and lets users reopen completed analyses through *`analytics.php?job_id=...`* without exposing server file paths.

Table 3.3: Main Web Portal Pages and Their Design Roles

Page	**Access Level**	**Design Role**	**Current Implementation Notes**
frontend/index.html	Public	Landing page and product introduction	Public entry point from /customer360/ through index.php redirect
frontend/register.php	Public	New user account creation	Uses PHP API proxy to call FastAPI registration and login routes
frontend/signin.php	Public	Existing user sign-in	Stores backend JWT in PHP session after successful login
frontend/dashboard.php	Authenticated	Job overview and shortcut navigation	Shows recent jobs and plain-language status wording
frontend/upload.php	Authenticated	CSV selection and upload initiation	Uses a non-bubbling Browse Files button and links to the RFM CSV template
frontend/column-mapping.php	Authenticated	Column mapping, parser rules, and	Required fields use visual markers, help popups, profile tables, and validation feedback

		validation preview	
frontend/processing.php	Authenticated	Live background job monitoring	Polls backend status through api/process.php?action=status
frontend/analytics.php	Authenticated	Final analysis dashboard and export hub	Hides export buttons when no result exists and supports in-card AI profiles
frontend/reports.php	Authenticated	Historical report and job browser	Displays safe base filenames only and links completed jobs to analytics
frontend/help.php	Authenticated	User support and plain-language explanation of the workflow	Explains the CSV-first guided mapping branch and the meaning of the buying fields

3.4 PHP Proxy, Session Management, and Authentication Design

The PHP layer handles front-end presentation and user interaction. During sign in, PHP sends the user credentials to FastAPI for verification. On successful authentication, FastAPI returns a JWT, which PHP stores in the session along with the user ID, user name, and company name. Protected pages are only accessible after checking $_SESSION['user_id'], while API proxy scripts use $_SESSION['auth_token'] as the bearer token for backend requests.

Table 3.4: PHP proxy scripts and FastAPI routes

Frontend Proxy Script	**Backend Route(s) Used**	**Design Rationale**
frontend/api/auth.php	/api/auth/register, /api/auth/login/json, /api/auth/me, /api/auth/logout	User registration, sign-in, session

		initialization, and logout
frontend/api/upload.php	/api/jobs/upload/preview, /api/jobs/upload	Preview upload and direct upload handling
frontend/api/mapping.php	/api/jobs/upload/validate-mapping, /api/jobs/upload/with-mapping	Validate mapping and parser rules, and launch mapped background jobs
frontend/api/process.php	/api/jobs/status/{job_id}, /api/jobs/results/{job_id}, /api/jobs/report/{job_id}, /api/jobs/download/{job_id}/customers, /api/jobs/chart/{job_id}/{chart_key}, /api/jobs/{job_id}	Progress polling, result retrieval, PDF, CSV, and chart streaming, as well as job deletion or cancellation
frontend/api/groq-insight.php	Groq Chat Completions API	On-demand segment profile generation for individual segment cards

3.4.1 Authentication and authorization sequence

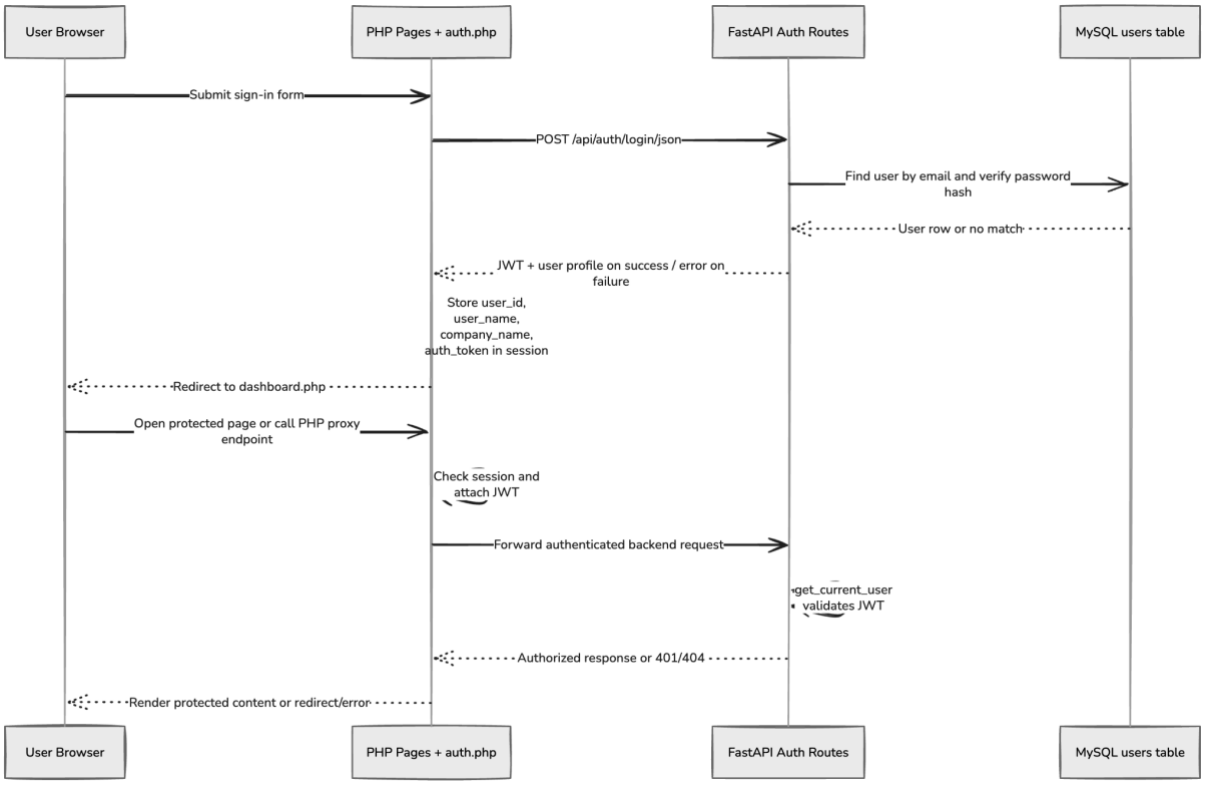


Figure 3.2: Authentication and Authorization Sequence[3]

3.5 FastAPI Job-Management Layer and Data Validation Design

The backend is responsible for generating preview metadata, validating column mappings and parser settings, creating background analysis on uploads, tracking job progress in MySQL, authorizing access to results, and serving output files. Most of this logic sits in *backend/app/routes/jobs.py.* Authentication is handled separately through *backend/app/auth.py* and *backend/app/routes/auth.py*. It is split into preview and validation first, then job creation. This matters because SME datasets rarely come in clean, column names differ, formats vary,

[3] For a higher-resolution version of this figure, click here.

and a bad mapping can affect an entire pipeline run. During preview, the backend reads a small sample of the CSV, profiles the columns, guesses likely mappings, and sends back sample rows with metadata. If the user adjusts anything, validation applies their mapping and parser rules, checks for problems, and returns a cleaned preview. The actual job only gets created once validation passes.

As the pipeline runs, it writes *progress_percent*, *progress_stage*, and *progress_message* to the job's row in MySQL. The processing page reads these fields and updates the progress bar accordingly. Every job lookup filters on both *job_id* and *current_user.id*, so a logged-in user cannot view someone else's results just by guessing a UUID.

Table 3.5: Core job lifecycle states and UI behavior

Job Status	**Where It Is Stored**	**What the User Sees**
pending	jobs.status,jobs.progress_stage, jobs.progress_message	The processing page shows that the job has been queued or is starting
processing	Same jobs row, updated by background callback	The processing page displays stage-by-stage backend progress
completed	jobs.status = completed, completed_at, summary fields, result cache	“View Results” becomes available and analytics.php?job_id=... loads the saved analysis
failed	jobs.status = failed, error_message, final progress message	The processing page shows a readable failure message and suggests retrying with corrected data
cancelled	jobs.status = cancelled	The job is stopped and no completed analytics view is exposed

3.5.1 Upload preview, mapping validation, and job creation sequence

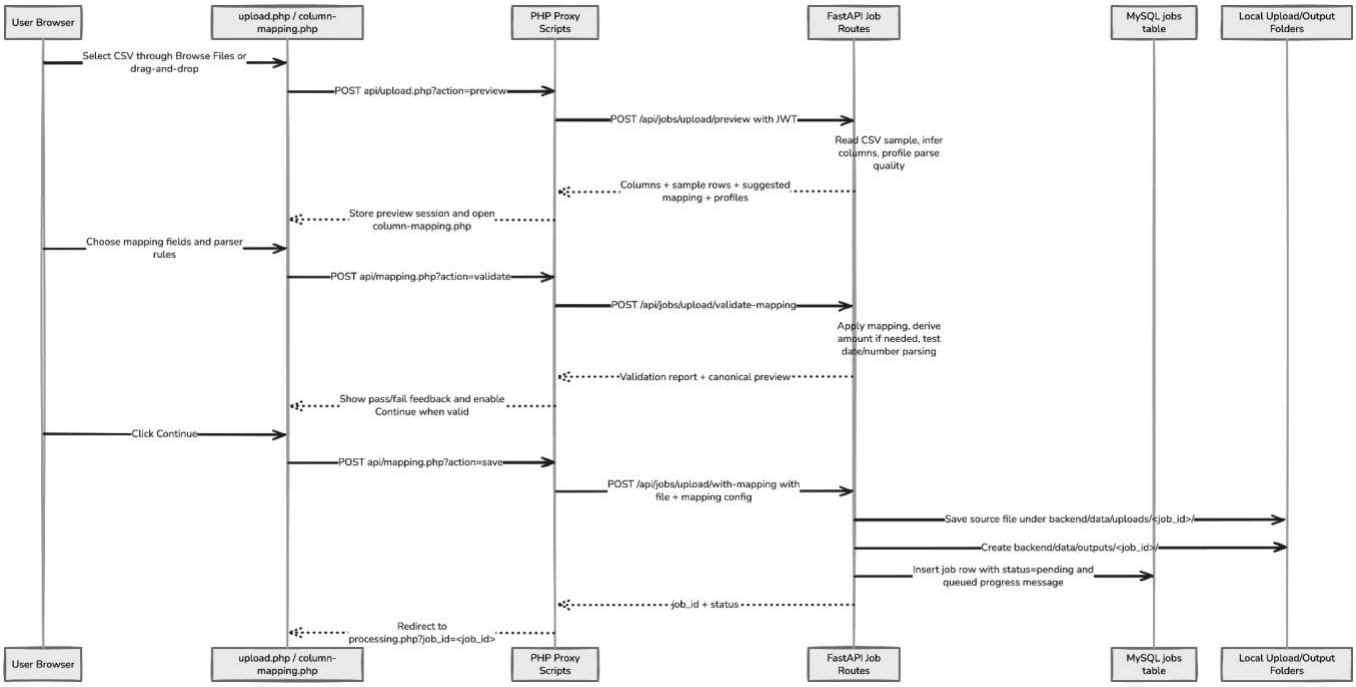


Figure 3.3: Upload Preview, Mapping Validation, and Job Creation Sequence[4]

3.6 Analytics Pipeline and Segmentation Design

The analytics pipeline is written in Python and stored in *backend/app/analytics/*. The main execution flow runs through pipeline.py, which calls into separate submodules for RFM computation, preprocessing, clustering, and segment interpretation. Each submodule handles one job, which makes the code easier to maintain and the processing order easy to follow.

[4] For a higher-resolution version of this figure, click here.

3.6.1 Data preprocessing and feature engineering

Once the mapped CSV passes validation, the pipeline cleans it in a set sequence: blank rows are removed before the row limit is enforced, mapped columns are renamed to the internal schema, dates are parsed based on the user's selected format and day-order settings, and numeric values are parsed according to the chosen decimal, thousands, and currency settings. If the user configured it, the pipeline derives *amount = quantity * unit_price.* Negative values are handled based on the policy the user selected. Any row still missing a usable customer ID, date, or monetary value gets dropped, unless the user explicitly allows a fallback.

From the cleaned transactions, the pipeline builds customer-level features using the RFM framework. Recency is the number of days since a customer's last purchase. Frequency is the count of distinct invoices or purchases tied to that customer. Monetary is their total spend. Frequency and Monetary tend to be heavily skewed, so both are log-transformed before clustering. All three variables are then scaled with StandardScaler so that no single feature dominates the clustering just because its numbers are larger.

3.6.2 Cluster-count selection and clustering-model comparison

The pipeline evaluates candidate cluster counts from 2 to 8 and selects the most defensible value of K through a majority-vote approach based on four internal evaluation measures: the Elbow heuristic, Silhouette Score, Calinski-Harabasz Index, and Davies-Bouldin Index. Once the final cluster count has been selected, the backend runs K-Means, Gaussian Mixture Model, and Hierarchical Clustering and chooses the method with the strongest Silhouette Score as the final clustering result. In the implemented web workflow, this comparison is enabled during normal analysis runs because the PHP upload and mapping

proxies submit include_comparison=true, meaning that all three algorithms are evaluated by default rather than only a single preselected method.

3.6.3 Customer-group naming and recommendation design

Raw cluster IDs are not shown to the user as the primary business result. Instead, each cluster is assigned a plain-language customer-group name from the segment definition library in `segmentation.py`, using the cluster's average Recency, Frequency, and Monetary profile. For example: 'Your Star Customers', 'Your Faithful Regulars', 'Almost Regulars', 'Fresh Faces', 'Slipping Away Slowly', 'Danger Zone', 'Sleeping Customers' , and 'Gone Customers'. Each group also receives a business explanation and suggested actions. If two clusters map to the same broad segment label, the pipeline appends suffixes such as Group 1 and Group 2 so that no two final displayed groups have exactly the same name.

3.6.4 Explainability, chart generation, and report artifacts

To explain which RFM metric most influenced the group assignment, the pipeline trains a Random Forest surrogate model on the scaled RFM features and uses SHAP to rank feature importance. The dashboard presents this in plain business language such as "how recently customers bought," "how often customers come back," and "how much customers spend."

The pipeline also generates static chart PNGs for the dashboard and PDF report, including the customer map, group-size chart, buying-pattern distribution charts, revenue Pareto chart, radar chart, model-comparison chart, and group-difference chart. The PDF report is generated with ReportLab and attaches these images alongside the executive summary,

segment tables, and recommended actions. Customer-level rows are exported to CSV for business use

3.6.5 Pipeline design diagram

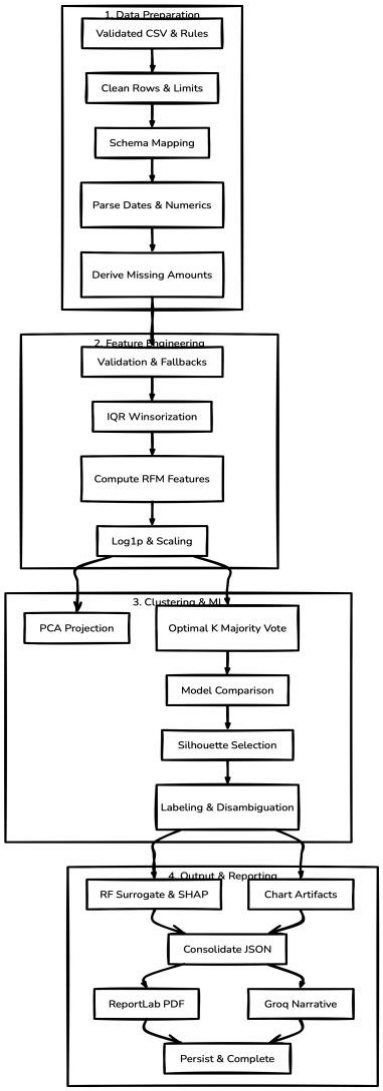


Figure 3.4:Pipeline Design Diagram[5]

[5] For a higher-resolution version of this figure, click here.

Table 3.6: Analytics Modules and Their Functions

Module	Responsibility in the System
preprocessing.py	CSV preview, column profiling, mapping validation, locale-aware parsing, blank-row removal, and amount derivation
rfm.py	Customer-level Recency, Frequency, and Monetary aggregation and normalization
clustering.py	Candidate K evaluation, K-Means, GMM, and Hierarchical execution, as well as model-comparison scoring
segmentation.py	Plain-language customer-group assignment and per-group action templates
pipeline.py	End-to-end orchestration, progress callbacks, chart generation, customer-table preparation, and output saving
groq_analysis.py	Optional aggregate-only AI business narrative generation with app-side rate limiting and fallback
report.py	ReportLab PDF generation with business summaries, tables, and embedded chart PNGs

3.7 Data Persistence, Storage, and Database Design

MySQL is used for user account management and job metadata storage. Larger files such as charts, PDF reports, and uploaded source datasets are not stored in the database but are written to per-job directories on the server filesystem. The database retains ownership records, job status, progress information, summary metrics, and cached AI narratives through the jobs table. The database schema consists of four tables: users, jobs, job_artifacts, and job_events. The users table stores account credentials and business identity fields. The jobs table holds one row per analysis run. Its columns are organized into several groups, including: identity and status (job_id, status, error_message), file references (original_filename, upload_path,

output_path), clustering configuration (clustering_method, include_comparison, column_mapping), summary metrics (num_customers, num_transactions, total_revenue, num_clusters, silhouette_score), progress tracking (progress_percent, progress_stage, progress_message), LLM narrative caching (llm_analysis_json, llm_status, llm_generated_at), and general housekeeping (created_at, completed_at, is_saved).

The frontend checks the progress fields during an analysis run to update a live progress bar, while the LLM columns cache the generated business narrative so that revisiting a results page does not trigger a repeated API call. job_artifacts and job_events are support tables — job_artifacts links artifact metadata to a job ID, and job_events logs timestamped events with JSON payloads.

3.7.1 Entity–Relationship Diagram

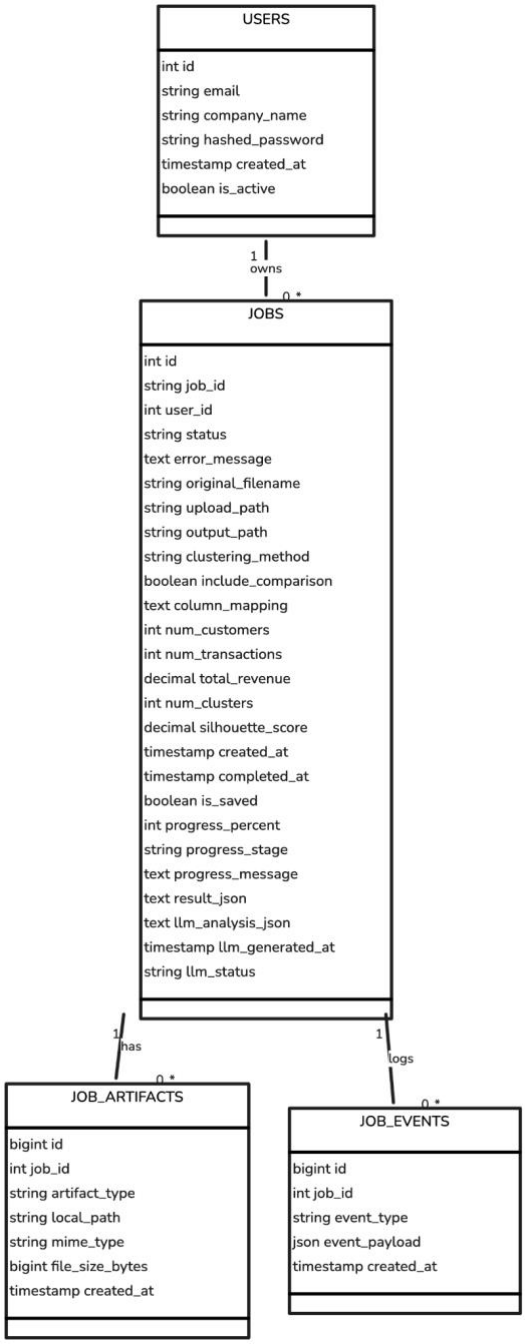


Figure 3.5: Entity–Relationship Diagram[6]

[6] For a higher-resolution version of this figure, click here.

3.8 Sequence Diagram: Upload Preview, Mapping, Validation, and Processing

This sequence diagram shows the primary workflow in the system, from CSV upload to a running analysis job which follows the preview-first flow as it moves through the PHP frontend and FastAPI backend

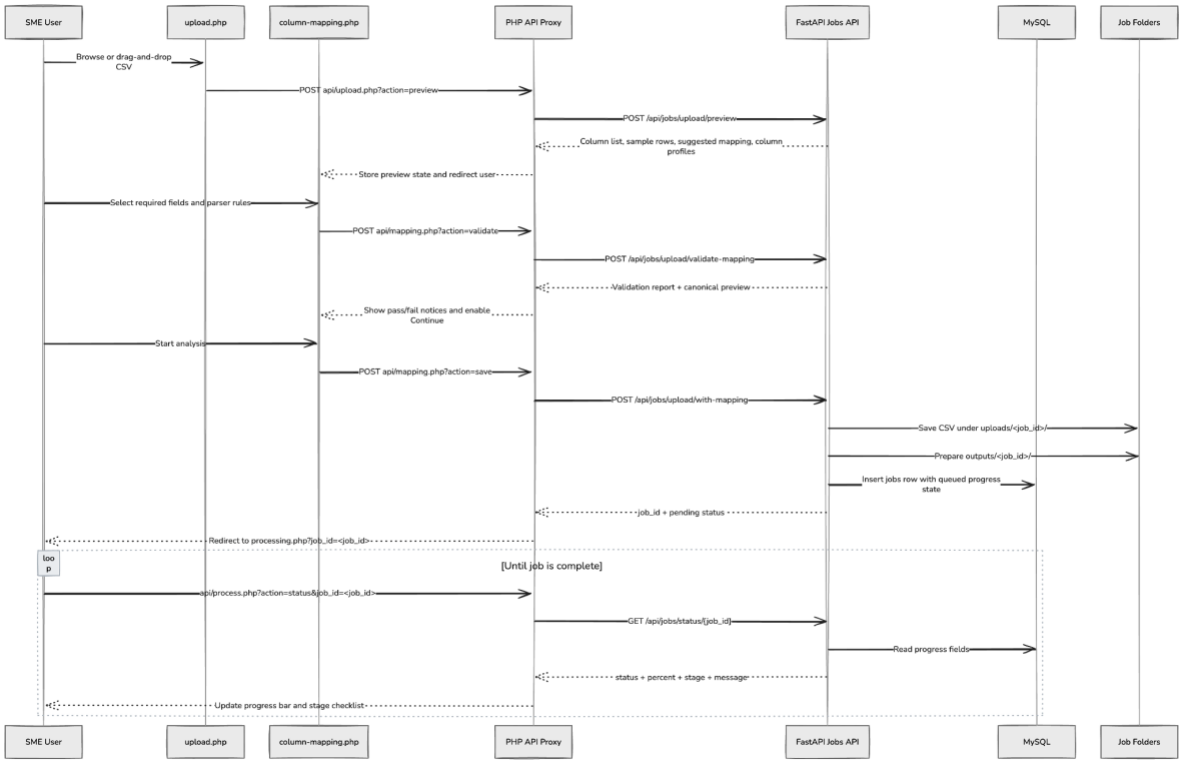


Figure 3.6: Upload Preview, Mapping, Validation, and Processing Sequence Diagram[7]

[7] For a higher-resolution version of this figure, click here.

3.9 Sequence Diagram: Post-Analysis Dashboard and Segment Narrative Flow

After a job analysis completes, *analytics.php* does not recompute the entire clustering pipeline. Instead, it requests the saved result payload for the selected *job_id,* renders the dashboard from that stored output, loads chart images through the PHP proxy, and optionally requests an extra per-segment AI profile when the user clicks the Get AI Profile button on a

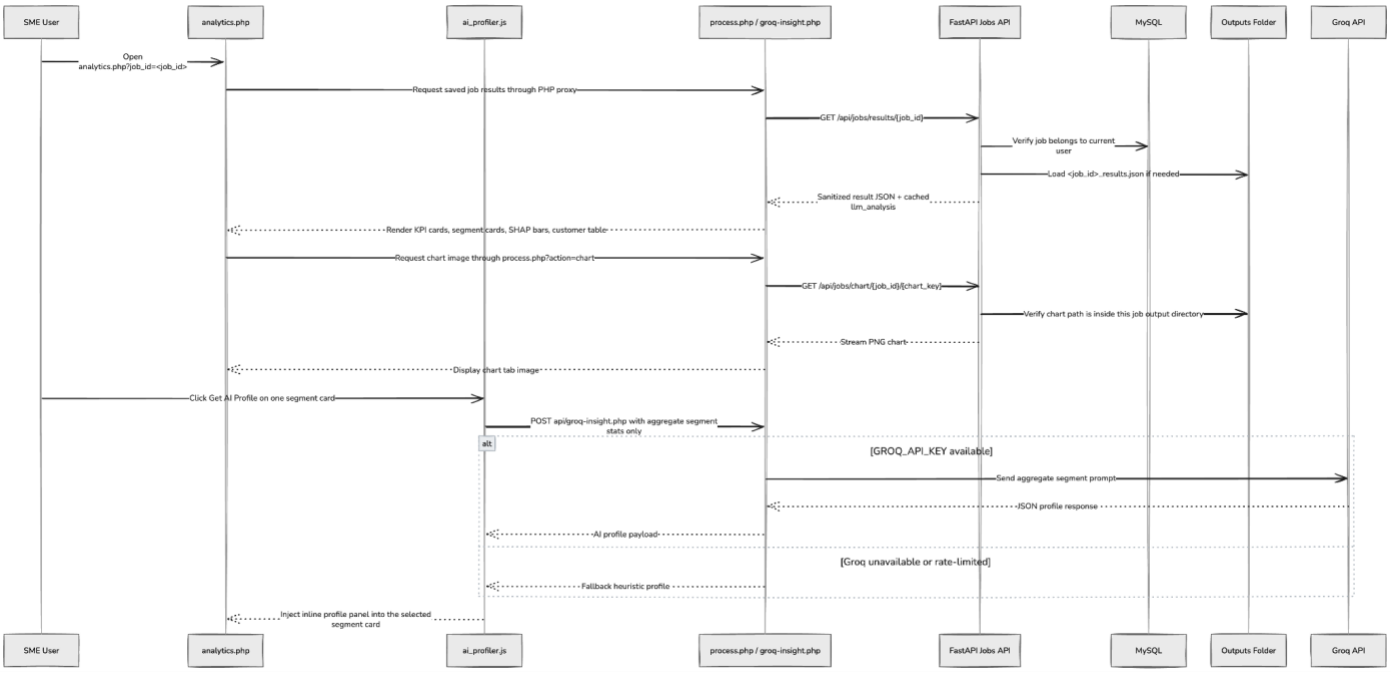
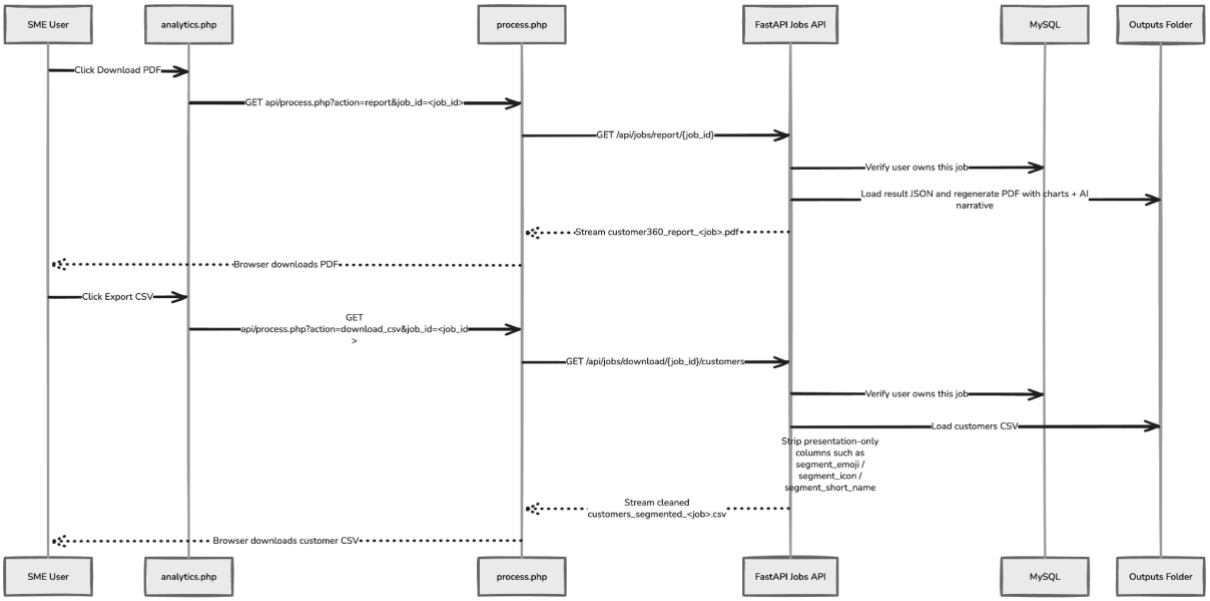


Figure 3.7: Sequence Diagram for the Post-Analysis Dashboard and Segment Narrative Flow[8]

[8] For a higher-resolution version of this figure, click here

3.10 Sequence Diagram: PDF and Segmented Customer CSV Download

This sequence diagram shows how the export actions work after analysis has completed. The browser never contacts FastAPI directly; all requests go through the PHP proxy, which forwards them to the backend. FastAPI then verifies job ownership, loads or regenerates the



requested file, and sends it back to the user.

Figure 3.8: Sequence Diagram for PDF and Segmented Customer CSV Download[9]

[9] For a higher-resolution version of this figure, click here.

3.11 Conclusion

This chapter has described the architecture and design of the current Customer360 system: a PHP frontend backed by a private FastAPI analytics service, MySQL for job metadata, and per-job server directories for generated outputs. The design supports the core project requirements : CSV upload and validation, live progress tracking, plain-language segmentation results, AI-assisted narratives, CSV export, and PDF reporting. Security is handled through session-gated pages, JWT authorization, localhost-only backend access, and per-job ownership checks. Background processing, cached result JSON, and modular code structure keep the system maintainable and avoid unnecessary recomputation. The chapters that follow cover the implementation details and testing of this design.

Chapter 4: Implementation

4.1 Introduction

This chapter explains how Customer360 was implemented as a working customer-segmentation platform for Ghanaian SMEs. The final implementation uses a hybrid server-side architecture in which Apache and PHP provide the public web interface, while a Python FastAPI service performs authentication, job execution, segmentation, explainability, file generation, and AI-assisted business narration. MySQL stores user and job metadata, while server-local directories store uploaded CSV files, generated charts, segmented customer exports, JSON result payloads, and PDF reports.

The implementation approach was influenced by three practical goals. The system had to be usable by non-technical SME operators through a guided web interface, not just a notebook. The analytical backend had to handle RFM preprocessing, multi-algorithm comparison, explainability, and reporting without exposing those details to the browser. And the deployment had to work reliably under real hosting constraints, which is why PHP and FastAPI were co-located on the same Ubuntu server and connected over localhost.

4.1.1 Implementation Philosophy and Justification of Major Choices

Each major implementation decision in this project was assessed in relation to usability, ease of deployment, analytical suitability, and cost of operation. Although the combined use of PHP and FastAPI is less common than adopting a single-stack architecture, this approach was maintained for three reasons. First, the analytical layer was written in Python because the libraries needed for data cleaning, RFM computation, clustering, explainability, and chart generation are better developed in Python than in PHP. Second, the user-facing workflow was

already built as a server-rendered PHP application with session-based navigation, which fit the sequential page flow (upload, mapping, processing, analytics, reporting) without needing a JavaScript frontend framework. In the deployed system, both services are colocated on the same Ubuntu server, with PHP proxying requests to FastAPI on 127.0.0.1:8000. This avoided cross-origin issues and kept the analytics backend off the internet

However, this design choice introduces some additional complexity, particularly in coordinating communication between two languages and handling session-token transfer across layers. In the current system, this is controlled by assigning clearly separated responsibilities to each part of the architecture. PHP is limited to page rendering, session management, and proxy communication, while FastAPI handles all analytical and data-processing tasks. As a result, PHP is not responsible for running analytics, and FastAPI is not used to render browser pages.

Table 4.1: Final Implementation Stack of the Deployed Customer360 System

System Layer	**Final Implementation**	**Role in the Deployed System**
Public Web Layer	Apache 2.4 + PHP	Serves web pages, manages PHP sessions, and proxies authenticated API requests
Frontend User Interface	PHP templates, Tailwind CSS, vanilla JavaScript, and Material Symbols	Renders the upload, mapping, processing, analytics, reports, and help pages
Application API Layer	FastAPI + Uvicorn	Handles authentication, upload preview, mapping validation, job control, results retrieval, and report and CSV delivery

Analytics Layer	pandas, NumPy, scikit-learn, SHAP, Plotly, Kaleido, and SciPy	Supports preprocessing, RFM generation, clustering, explainability, and chart generation
Database Layer	MySQL	Stores users, jobs, cached job data, and database-side audit and artifact tables
File Storage Layer	Server-local directories under backend/data/	Stores uploads, chart PNGs, segmented CSVs, PDF reports, and results JSON
Optional AI Narrative Layer	Groq API	Generates plain-language summaries and recommendations from aggregate statistics only

4.1.1 User Workflows

The final solution supports two main ways of use: a guided web workflow for SME users and a notebook-based analytical workflow for researchers or analysts. The web workflow is the main deployed version of the system, while the notebook workflow remains useful for experimentation, step-by-step inspection, and academic analysis

Workflow A: SME business owner using the web application

1. The user opens the public Customer360 site served by Apache.

2. The user registers or signs in through PHP pages, which forward credentials to FastAPI and store the returned token in the PHP session.

3. The user uploads a CSV file through the upload page.

4. The system previews the file, extracts available columns, and routes the user to the column-mapping page.

5. The user maps business fields such as customer identifier, invoice date, invoice number, amount, quantity, and unit price, then validates parsing rules.

6. The mapped dataset is submitted as a background job.

7. The processing page polls job status through PHP, while FastAPI updates progress and stage messages in MySQL.

8. When the job completes, the user opens `*analytics.php?job_id=...*` to view KPI cards, customer groups, AI-assisted summaries, charts, feature-importance explanations, and the customer table.

9. The user can then export a segmented customer CSV or download a PDF report for that job.

Workflow B: Researcher or Analyst Using the Notebook Pipeline

1. The researcher or analyst uses the Python or Colab version of the pipeline for step-by-step analysis.

2. The dataset is loaded directly into Python instead of being uploaded through the PHP interface.

3. The notebook is then used to carry out preprocessing, RFM generation, cluster selection, algorithm comparison, and explainability.

4. This workflow remains useful for experimentation, academic comparison, and closer inspection of parameters, but it is not the main deployed user path.

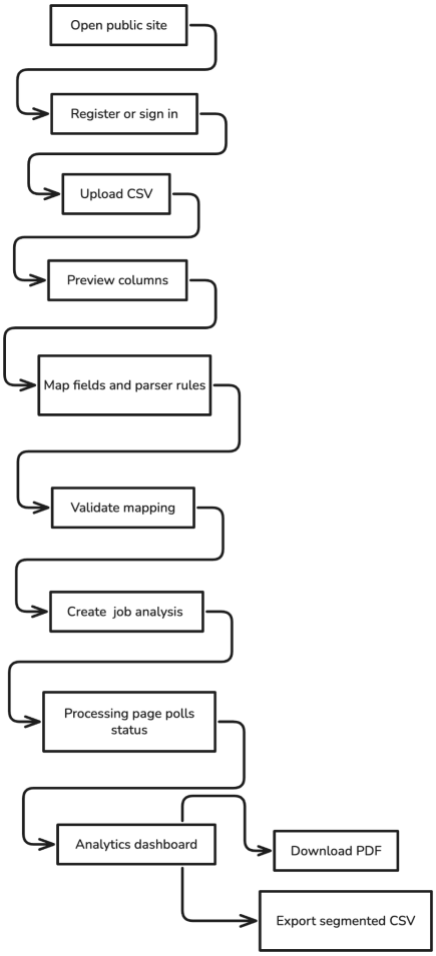


Figure 4.1: Final Web Workflow Implementation[10]

4.2 Tools and Technologies

The implementation uses a mixed technology stack for its requirements. PHP runs the web layer, handling page rendering, session management, and the step-by-step user workflow. Python runs the analytical pipeline, covering data cleaning, RFM computation, clustering, algorithm comparison, SHAP analysis, and chart generation. PHP was kept for the frontend because it managed page rendering and user sessions. Python was kept for the backend because the libraries required for the analytical work have no practical equivalent in PHP.

[10] For a higher-resolution version of this figure, click here.

Table 4.2: Tools and Technologies Used Across System Layers

System Layer	Tools and Frameworks	Purpose in the Current System
Web Server Layer	Apache 2.4	Provides stable hosting for PHP pages and PHP proxy endpoints
Frontend Rendering	PHP, Tailwind CSS, vanilla JavaScript	Supports a lightweight server-rendered interface with minimal browser-side complexity
Iconography	Material Symbols	Provides a clear icon set after emoji-based labels were removed
API and Backend Layer	FastAPI, Uvicorn, Pydantic, SQLAlchemy	Supports structured API design, validation, background processing, and database integration
Database Layer	MySQL	Stores users, jobs, progress data, and cached AI fields
Data Processing	pandas, NumPy	Handles CSV parsing, data cleaning, and feature engineering
Machine Learning	scikit-learn, SciPy	Supports scaling, clustering, evaluation metrics, and surrogate model training
Explainability	SHAP	Explains which buying signals influence group formation
Charts and Export Visuals	Plotly, Kaleido	Produces PNG charts used in analytics pages and PDF reports
Reporting	ReportLab	Generates PDF reports containing charts, summaries, and recommendations
AI Narrative	Groq Python SDK	Produces optional plain-language business summaries from aggregate statistics
Deployment and Runtime	Python virtual environment, system	Supports managed FastAPI service deployment on the Ubuntu server

Table 4.2 A: Justification of the main implementation choices

Implementation Choice	**Reason for Use in the Project**	**Trade-off**
Apache + PHP for the public site	Server-rendered pages and session handling fit the guided SME workflow well and are straightforward to host on the target server	Adds an extra proxy layer between the browser and FastAPI
FastAPI for the analytical backend	Best suited to Python-based analytics, validation, background processing, and structured API routing	Requires explicit integration with the PHP layer
PHP proxy to localhost FastAPI	Keeps browser traffic on a single origin, reduces backend exposure, and avoids unstable public-IP or laptop-hosted backends	Introduces more moving parts than a single-stack application
MySQL for metadata	Provides stable relational storage for users, jobs, progress records, and cached results	Schema changes must be managed carefully as new fields are introduced
Server-local file storage	Provides reliable storage for generated artifacts and is easier to manage than unfinished external mirroring	Requires server disk management and clear retention policies
CSV-first mapped workflow	Gives more predictable parsing and validation for SME datasets than allowing many spreadsheet formats in the main guided flow	Limits the range of file types accepted in the main web workflow
Multi-algorithm comparison	Reduces the risk of relying on one unsuitable clustering method for every dataset	Increases runtime compared with a single-algorithm pipeline

Groq as optional post-processing	Improves readability for SME users without making AI availability a core system dependency	Adds reliance on an external service for enhanced narrative output only

4.2.1 Technologies Considered but Not Retained in the Final System

Several implementation options changed over the course of the project. In each case, the final system adopted the option that was simpler to operate reliably within the project's hosting, usability, and cost constraints.

Table 4.3: Technologies Retained, Narrowed, or Excluded in the Final System

Technology or Approach	**Final Outcome**	**Rationale**
FastAPI backend hosted separately from PHP	Discontinued	The final deployment is more reliable when FastAPI runs privately on the same Ubuntu host as Apache and PHP
Mac-hosted development backend for production calls	Discontinued	Public IP changes, NAT issues, and unstable reachability made it unsuitable for production use
Supabase upload mirroring	Discontinued from the final upload flow	The final system stores uploads and outputs locally on the server for reliability and simpler operational control
Excel/XLSX web upload path in the guided mapping flow	Narrowed	The final mapped web workflow is CSV-first to reduce parsing ambiguity and improve more predictable validation
Direct browser-to-FastAPI communication	Not used in the deployed web flow	PHP proxy scripts mediate authenticated requests and keep the backend private

Groq as a core analysis dependency	Not adopted	Groq remains optional and non-blocking, so the pipeline can still complete successfully without it.

4.3 Implementation Techniques

4.3.1 Model and algorithm development

The implementation remains centered on transaction-based RFM segmentation. The analytical logic begins with dataset profiling and controlled field mapping rather than assuming a rigid schema. After upload preview, the user selects which CSV columns match certain fields such as customer ID, invoice date, invoice ID, amount, quantity, unit price, product, and category. Parsing rules such as decimal separator, thousands separator, date-format hints, and negative-value policy are also captured before job creation.

The preprocessing stage then:

- Drops fully blank rows before row-limit enforcement
- Normalizes column names based on the user mapping
- Derives `amount = quantity × unit_price` when direct amount is not provided
- Removes rows that cannot support valid RFM construction
- Aggregates transaction rows to the customer level

RFM features are then generated as:

1. *Recency*: days since the customer's most recent valid purchase
2. *Frequency*: count of transactions or invoice events for that customer
3. *Monetary*: cumulative spend after cleaning and amount derivation

In the implemented web workflow, the system compares multiple clustering choices by default. It first evaluates candidate values of K from 2 to 8 and uses majority voting across the

Elbow heuristic, Silhouette Score, Calinski-Harabasz Index, and Davies-Bouldin Index to select the most defensible cluster count. It then runs K-Means, Gaussian Mixture Model, and Hierarchical Clustering for that selected K and chooses the final grouping based on the strongest Silhouette Score among the compared methods.

Table 4.4: Implemented analytical stages

Analytical Stage	**Implementation Detail**	**Main Output**
Upload Preview	CSV inspection and column extraction	Preview metadata
Mapping Validation	Parser-rule and required-field validation	Validation report
Preprocessing	Cleaning, date parsing, numeric normalization, and amount derivation	Clean transaction table
Customer Aggregation	RFM feature construction	Customer-level RFM table
K Selection	Majority vote across four internal metrics for K = 2- 8	Chosen cluster count
Algorithm Comparison	Comparison of K-Means, GMM, and Hierarchical clustering	Final grouping method
Segment Naming	Business-friendly group labels and recommended actions	Segment definitions
Explainability	Random Forest surrogate model and SHAP importance analysis	Feature-importance summary
Output Generation	Charts, results JSON, segmented customer CSV, and PDF report generation	Downloadable artifacts

Table 4.5: Implemented evaluation metrics

Evaluation Metric	**Current Use in the Implementation**
Elbow Method	Contributes to cluster-number selection
Silhouette Score	Used in both cluster-number selection and final algorithm comparison
Calinski–Harabasz Index	Contributes to cluster-number selection
Davies–Bouldin Index	Contributes to cluster-number selection

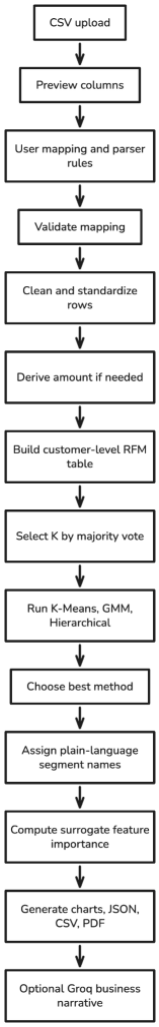


Figure 4.2: Implemented Analysis Pipeline[11]

4.3.2 Backend API and application implementation

The Customer360 backend is implemented in FastAPI and provides the main service-layer functionality of the system. Its responsibilities include user authentication, upload preview, column-mapping validation, background job creation, progress tracking, result

[11] For a higher-resolution version of this figure, click here.

retrieval, chart delivery, segmented CSV export, and PDF report generation. However, in the deployed web workflow, the user's browser does not communicate directly with the FastAPI service. Instead, PHP proxy scripts located under frontend/api/ handle browser requests, read the authenticated PHP session, and forward requests to FastAPI using a stored JWT token.

This design keeps browser traffic within the Apache-hosted PHP layer, while the analytical backend remains private on the same server. In the final deployment, Apache, PHP, and FastAPI are colocated on the same Ubuntu server where the PHP layer forwards requests internally to FastAPI via 127.0.0.1:8000 rather than through a public network route. This reduces cross-origin complexity and simplifies token forwarding and access control. This architectural design choice balances usability, security, deployment stability, and analytical capability within the constraints of the project

Table 4.6: Main Implementation Modules and Their Function

Module	**Function**
frontend/api/auth.php	Registration, login, logout, and session-based authentication checks
frontend/api/upload.php	Upload preview and file forwarding
frontend/api/mapping.php	Mapping validation and mapped job submission
frontend/api/process.php	Job-status polling, chart streaming, CSV export, and PDF delivery
backend/app/routes/auth.py	FastAPI authentication endpoints
backend/app/routes/jobs.py	Job creation, validation, status tracking, results retrieval, and chart, report, and download endpoints
backend/app/analytics/pipeline.py	End-to-end segmentation pipeline

backend/app/analytics/groq_analysis.py	Optional AI narrative generation and rate-limited Groq integration
backend/app/report.py	PDF report generation

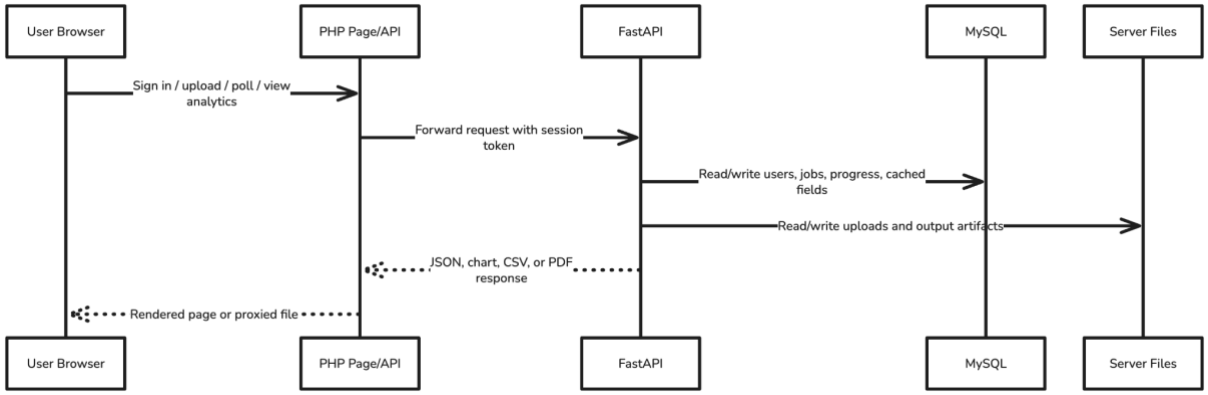


Figure 4.3: Implemented Request Chain[12]

4.3.3 Database design

The Customer360 database supports user management, job tracking, progress monitoring, result persistence, and artifact management. MySQL is used for structured application data, while larger files such as uploaded datasets, JSON results, chart images, segmented CSVs, and PDF reports are stored in job-specific server directories. This separation allows efficient metadata management without storing large files in relational tables.

At the application level, the FastAPI ORM models two core tables: users and jobs. The users table manages authentication and ownership, while the jobs table stores one record per analysis run and acts as the central operational table. It includes job ownership, uploaded filename, processing status, progress fields, clustering configuration, summary metrics, cached

[12] For a higher-resolution version of this figure, click here.

results, and AI narrative content, enabling background processing, access control, and result reuse without rerunning the pipeline.

The MySQL schema also includes auxiliary tables job_artifacts and job_events. The job_artifacts table is intended to record metadata about generated files associated with each job, such as uploads, reports, CSV exports, and result files while job_events tracks job lifecycle activity for operational and audit purposes. A relational database is appropriate for this design because Customer360 is a job-oriented and stateful application. MySQL therefore provides a suitable foundation for maintaining consistency, traceability, and controlled access across the implemented workflow.

Table 4.7: Current database entities in the deployed schema

Database Table	**Role in the Deployed System**
users	Stores registered user accounts and identity information
jobs	Stores upload job metadata, status, progress details, file references, cached results, and AI narrative fields
job_artifacts	Stores database-level metadata and file paths for generated artifacts
job_events	Stores event logs and audit-style tracking information for jobs

4.3.4 Web Application Implementation Details

The frontend implementation follows a multi-page web design, rather than as a single-page frontend. This design is an appropriate choice because workflow is sequential and task-

based, beginning with account access and then moving through file upload, column mapping, validation, processing, analytics review, and report retrieval

The publicly accessible pages are developed in PHP and served through Apache. These include the landing page, registration page, and sign-in page, while the authenticated pages support the main operational workflow of the system. After signing in, users can access the dashboard, upload page, column-mapping page, processing page, analytics page, reports page, and help page. The upload page handles file submission, the column-mapping page supports field assignment and parser-rule configuration, the processing page shows backend progress updates, the analytics page displays completed segmentation results, and the reports page allows previously completed jobs to be reopened.

This approach was chosen because it aligns with user needs and the workflow structure. While a single-page frontend was possible, it would require more client-side state management, direct browser interaction with backend endpoints, and a more complex security model involving tokens and session continuity. The current page-driven structure is easier to host, easier to secure through PHP session checks, and better suited to a guided step-by-step experience for SME users who may not have advanced technical skills.

Table 4.8: Main Web Pages Implemented in the Current System

Web Page	**Implementation Rationale**
landingpage.php	Serves as the public landing page
register.php, signin.php	Provide public entry points for account registration and sign-in
upload.php	Supports file selection and access to the upload template

column-mapping.php	Handles required-field mapping, validation, and parser-rule configuration
processing.php	Displays live progress polling and completion handoff
analytics.php	Serves as the main results dashboard
reports.php	Displays historical completed analyses
help.php	Provides user guidance and support

4.3.5 AI narrative and segment-profiling implementation

The final implementation includes two separate AI-assisted layers. The first being cached Groq-generated business summary created once per completed job. This uses only aggregate segment and quality statistics and is stored in the results payload and job record. It enhances the analytics page and PDF report but does not block the initial job analysis process. The second layer represents the on-demand segment profiler on `analytics.php`, where each segment card can request an additional plain-language profile through `ai_profiler.js`. This uses page-level aggregate segment metadata rather than raw customer rows and is designed to keep the language useful for non-technical Ghanaian SME operators.

4.4 Conclusion

This chapter described the implementation of Customer360 as a working hybrid system combining an Apache/PHP web interface, a FastAPI analytical backend, a MySQL metadata store, server-local file storage, a multi-stage RFM and clustering pipeline, explainability outputs, downloadable reports, and optional Groq-powered business summaries

Several design decisions were narrowed during implementation to fit reliability and usability constraints. The web workflow became CSV-first, server-local storage replaced external mirroring for uploads, the backend was colocated with Apache to avoid unstable remote calls, and technical language was reduced across the user-facing output. These were practical trade-offs that improved stability, reduced user confusion, and made the system more realistic as an SME-facing platform. The next chapter evaluates the system as a working and deployable solution, examining its testing, performance, limitations, and areas for future improvement.

Chapter 5: Testing and Results

5.1 Introduction

This chapter presents the testing conducted on Customer360 and the results obtained. The goal was to ensure that the system met its primary functional requirements, that the integrated process worked properly from file upload to final output generation, and that the delivered prototype produces stable and usable analytical results. Considering Customer360 incorporates a PHP online interface, a FastAPI analytical backend, MySQL-based metadata storage, and produced PDF and CSV outputs, testing was done at both the component and system levels. Testing is organized into three areas: component testing, system-level testing, and user acceptance testing indicating that the system performs as a full customer segmentation platform, rather than separate components.

5.2 Component Testing

Component testing focused on the analytical and backend logic implemented in Python. The objective was to verify that core processing units behaved efficiently independently before being tested through the full web workflow. The component tests covered: preprocessing and data cleaning, RFM feature engineering, clustering and algorithm comparison logic backend and API route behavior. Each area has a dedicated test file in the repository (*test_rfm.py, test_clustering.py, test_preprocessing.py, and test_api.py*). The automated backend suite was rerun on the current codebase and produced a complete pass.

Table 5.1: Summary of Automated Component Test Results

Test Metric	**Result**
Total automated tests collected	53
Total tests passed	53
Total tests failed	0
Test runtime	10.27 seconds
Test environment	Python 3.11.4, pytest 7.4.4

```
DATABASE_URL=sqlite:////tmp/customer360_test.db ENVIRONMENT=development DEBUG=1 pytest tests/ -v

The terminal output I confirmed ends with:

collected 53 items

tests/test_api.py::TestAuthEndpoints::test_register_user_success PASSED
tests/test_api.py::TestAuthEndpoints::test_register_user_duplicate_email PASSED
tests/test_api.py::TestAuthEndpoints::test_register_user_invalid_email PASSED
tests/test_api.py::TestAuthEndpoints::test_register_user_short_password PASSED
tests/test_api.py::TestAuthEndpoints::test_login_success PASSED
tests/test_api.py::TestAuthEndpoints::test_login_wrong_password PASSED
tests/test_api.py::TestAuthEndpoints::test_login_nonexistent_user PASSED
tests/test_api.py::TestAuthEndpoints::test_get_current_user PASSED
tests/test_api.py::TestAuthEndpoints::test_get_current_user_no_token PASSED
tests/test_api.py::TestJobEndpoints::test_list_jobs_empty PASSED
tests/test_api.py::TestJobEndpoints::test_list_jobs_unauthorized PASSED
tests/test_api.py::TestJobEndpoints::test_upload_file_no_auth PASSED
tests/test_api.py::TestJobEndpoints::test_upload_invalid_file_type PASSED
tests/test_api.py::TestJobEndpoints::test_get_job_status_not_found PASSED
tests/test_api.py::TestJobEndpoints::test_cancel_job_success PASSED
tests/test_api.py::TestJobEndpoints::test_cancel_completed_job_rejected PASSED
tests/test_api.py::TestHealthEndpoint::test_health_check PASSED
tests/test_api.py::TestHealthEndpoint::test_root_endpoint PASSED
tests/test_clustering.py::TestFindOptimalK::test_find_optimal_k_returns_valid_result PASSED
tests/test_clustering.py::TestFindOptimalK::test_find_optimal_k_detects_clusters PASSED
tests/test_clustering.py::TestKMeans::test_run_kmeans_returns_labels PASSED
tests/test_clustering.py::TestKMeans::test_run_kmeans_returns_info PASSED
tests/test_clustering.py::TestKMeans::test_run_kmeans_silhouette_positive PASSED
tests/test_clustering.py::TestGMM::test_run_gmm_returns_labels PASSED
tests/test_clustering.py::TestGMM::test_run_gmm_returns_info PASSED
tests/test_clustering.py::TestHierarchical::test_run_hierarchical_returns_labels PASSED
tests/test_clustering.py::TestHierarchical::test_run_hierarchical_returns_info PASSED
tests/test_clustering.py::TestRunClustering::test_run_clustering_auto_k PASSED
tests/test_clustering.py::TestRunClustering::test_run_clustering_specified_k PASSED
tests/test_clustering.py::TestRunComparison::test_run_comparison_all_methods PASSED
tests/test_clustering.py::TestRunComparison::test_run_comparison_consistent_labels PASSED
tests/test_preprocessing.py::TestLoadCSV::test_load_csv_basic PASSED
tests/test_preprocessing.py::TestLoadCSV::test_load_csv_with_encoding PASSED
tests/test_preprocessing.py::TestColumnMapping::test_suggest_column_mapping_exact_match PASSED
tests/test_preprocessing.py::TestColumnMapping::test_suggest_column_mapping_case_insensitive PASSED
tests/test_preprocessing.py::TestColumnMapping::test_suggest_column_mapping_partial_match PASSED
tests/test_preprocessing.py::TestCleanData::test_clean_data_removes_nulls PASSED
tests/test_preprocessing.py::TestCleanData::test_clean_data_returns_dataframe PASSED
tests/test_preprocessing.py::TestCleanData::test_clean_data_preserves_valid_rows PASSED
tests/test_preprocessing.py::TestCleanData::test_clean_data_min_max_amounts PASSED
tests/test_rfm.py::TestComputeRFM::test_compute_rfm_basic PASSED
tests/test_rfm.py::TestComputeRFM::test_compute_rfm_recency_values PASSED
tests/test_rfm.py::TestComputeRFM::test_compute_rfm_frequency_values PASSED
tests/test_rfm.py::TestComputeRFM::test_compute_rfm_monetary_values PASSED
tests/test_rfm.py::TestComputeRFM::test_compute_rfm_empty_dataframe PASSED
tests/test_rfm.py::TestComputeRFM::test_compute_rfm_no_negative_recency PASSED
tests/test_rfm.py::TestComputeRFMScores::test_compute_rfm_scores_columns PASSED
tests/test_rfm.py::TestComputeRFMScores::test_compute_rfm_scores_range PASSED
tests/test_rfm.py::TestComputeRFMScores::test_compute_rfm_scores_total PASSED
tests/test_rfm.py::TestNormalizeRFM::test_normalize_rfm_standard PASSED
tests/test_rfm.py::TestNormalizeRFM::test_normalize_rfm_minmax PASSED
tests/test_rfm.py::TestRFMStatistics::test_get_rfm_statistics PASSED
tests/test_rfm.py::TestRFMStatistics::test_get_rfm_distributions PASSED

============================= 53 passed in 5.09s ==============================
```

Figure 5.1:Terminal Output Showing Successful Execution of the Customer360 Backend Test Suite

Table 5.2: Component testing coverage

Component Area	What Was Verified	Result
Preprocessing	Cleaning, column mapping, invalid-input handling, and safe row filtering	Passed
RFM feature engineering	Customer-level recency, frequency, and monetary calculations	Passed
Clustering logic	Valid clustering outputs and comparison behavior	Passed
Backend API routes	Health checks and selected route behavior through FastAPI tests	Passed

The component-testing results provide strong evidence that the analytical core of the system is functioning correctly before deployment-level integration is considered.

5.3 System-Level Testing

System-level testing was used to verify the complete workflow of the deployed application. This involved checking that the PHP frontend, FastAPI backend, MySQL metadata layer, and generated outputs work together correctly in the hosted environment.

The tested workflow included:

1. Access to the public landing page
2. User sign-in
3. CSV upload
4. Preview and column mapping
5. Mapping validation
6. Job creation and progress monitoring
7. Analytics result retrieval

8. PDF report download

9. Segmented customer CSV export

10. Access to historical reports

Table 5.3: System-level workflow results

Test Area	**Expected Result**	**Result**
Public route handling	The root route should resolve to the application landing page	Passed
Authentication workflow	Valid users should be able to sign in and access protected pages	Passed
Upload and preview	CSV upload should return preview metadata and detected columns	Passed
Column mapping	Users should be able to map required fields and validate parser rules	Passed
Job creation	A valid mapping should create an analysis job with a job ID	Passed
Progress tracking	The processing page should display backend-driven job progress	Passed
Analytics retrieval	Completed jobs should open correctly in analytics.php	Passed
PDF reporting	Completed jobs should return a generated PDF report	Passed
CSV export	Completed jobs should return a cleaned segmented customer CSV	Passed
Historical retrieval	Users should be able to reopen completed jobs from the reports page	Passed

Live deployment checks confirmed that the backend service and integration path work as expected.

Table 5.4: Deployment verification results

Deployment Check	**Result**
FastAPI /health endpoint	Healthy response returned
Customer360-backend service state	Active
PHP-to-FastAPI proxying	Working
PDF generation after deployment fixes	Working
CSV export after sanitization fixes	Working

The following figures present evidence from system-level testing conducted on the deployed platform. Each figure captures a stage of the end-to-end workflow, from landing page through to final report generation, and confirms that the corresponding functional requirement is met.

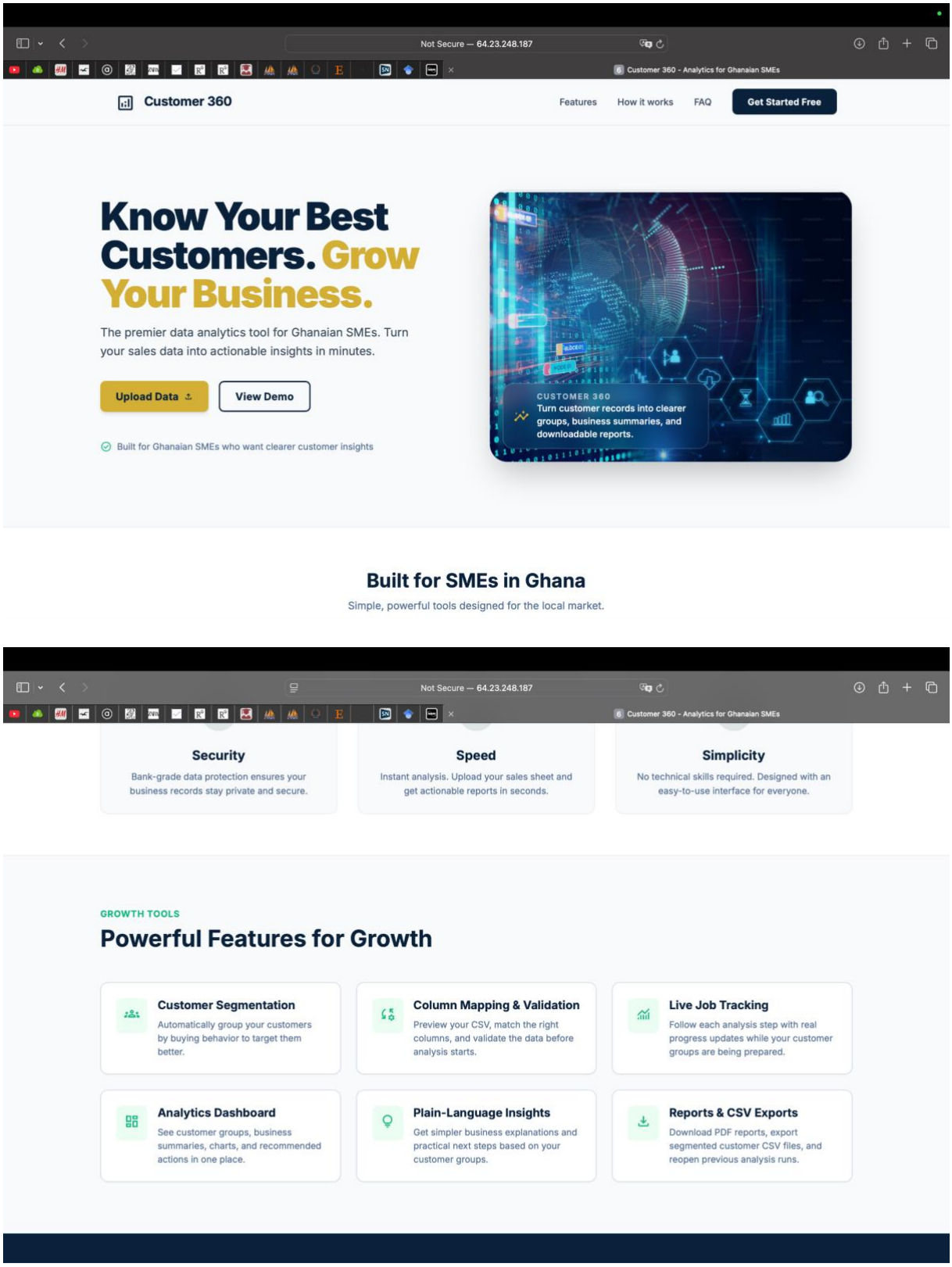


Figure 5.2,5.3: Public landing page of the Customer360 system.

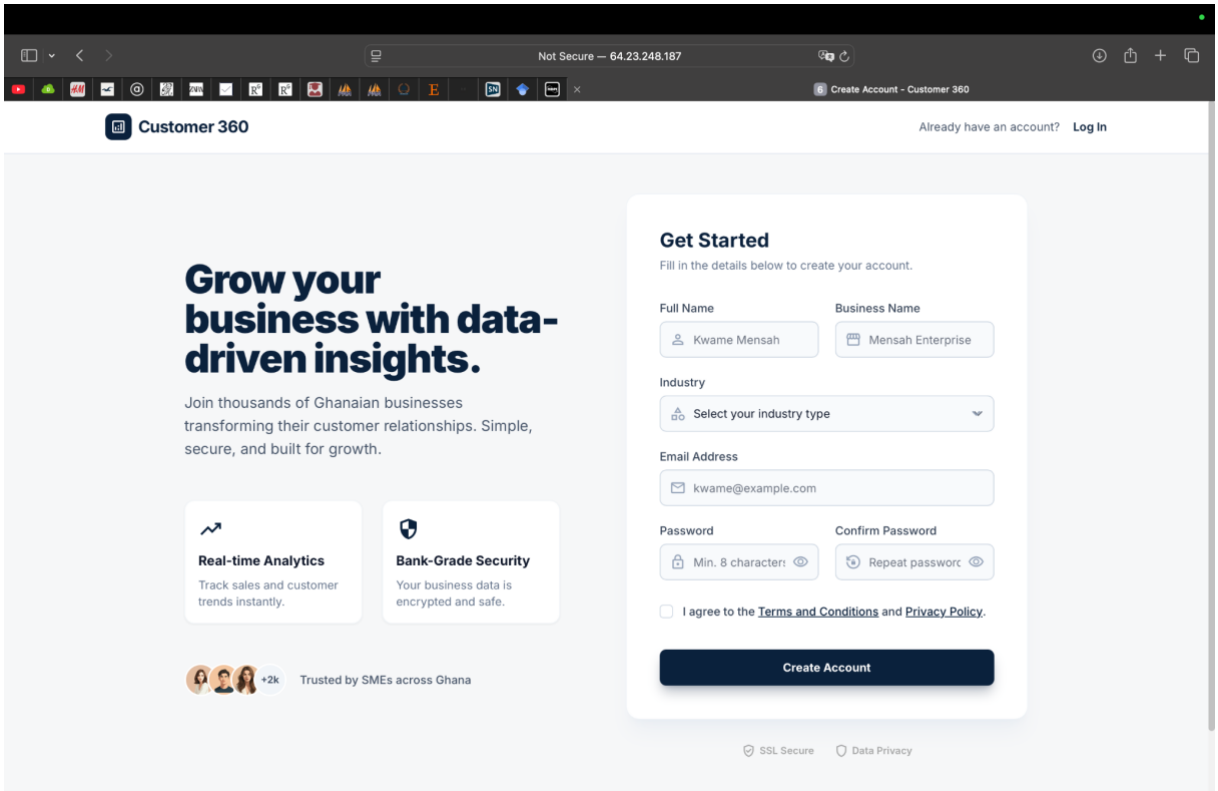


Figure 5.4: Sign-in page for authenticated access.

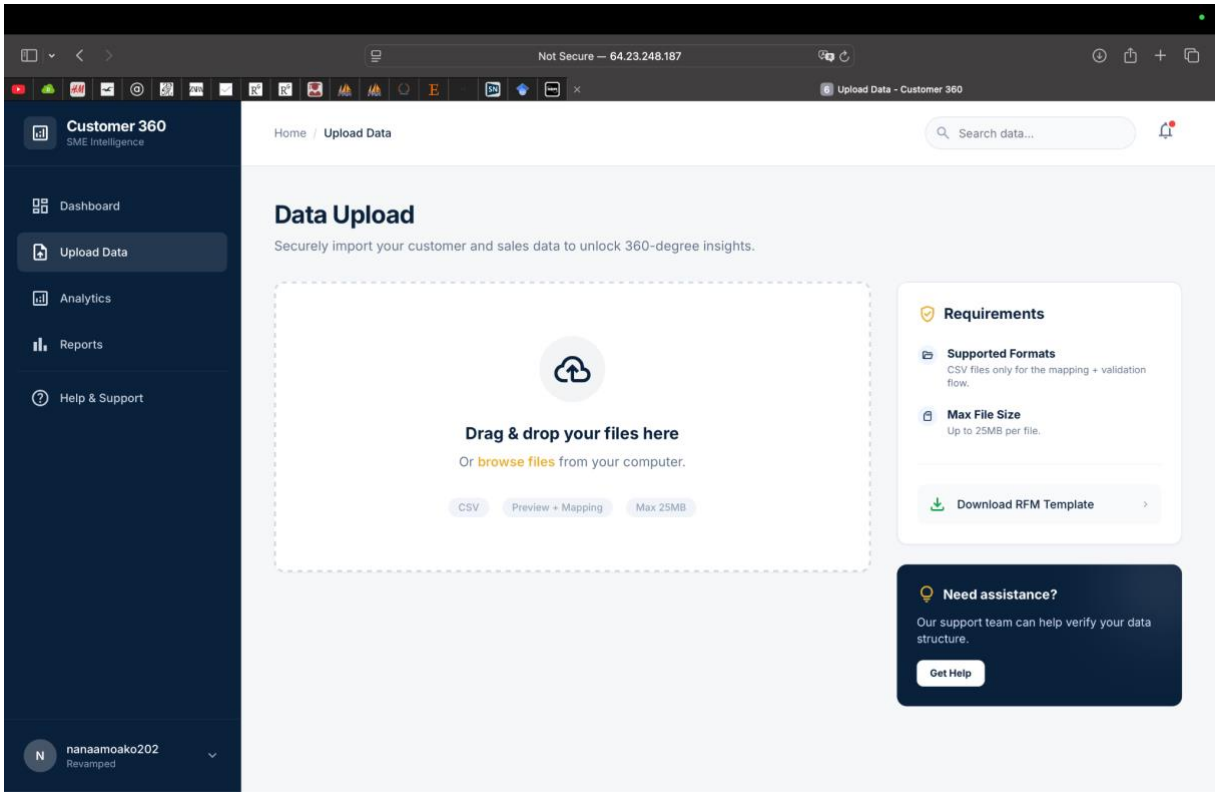
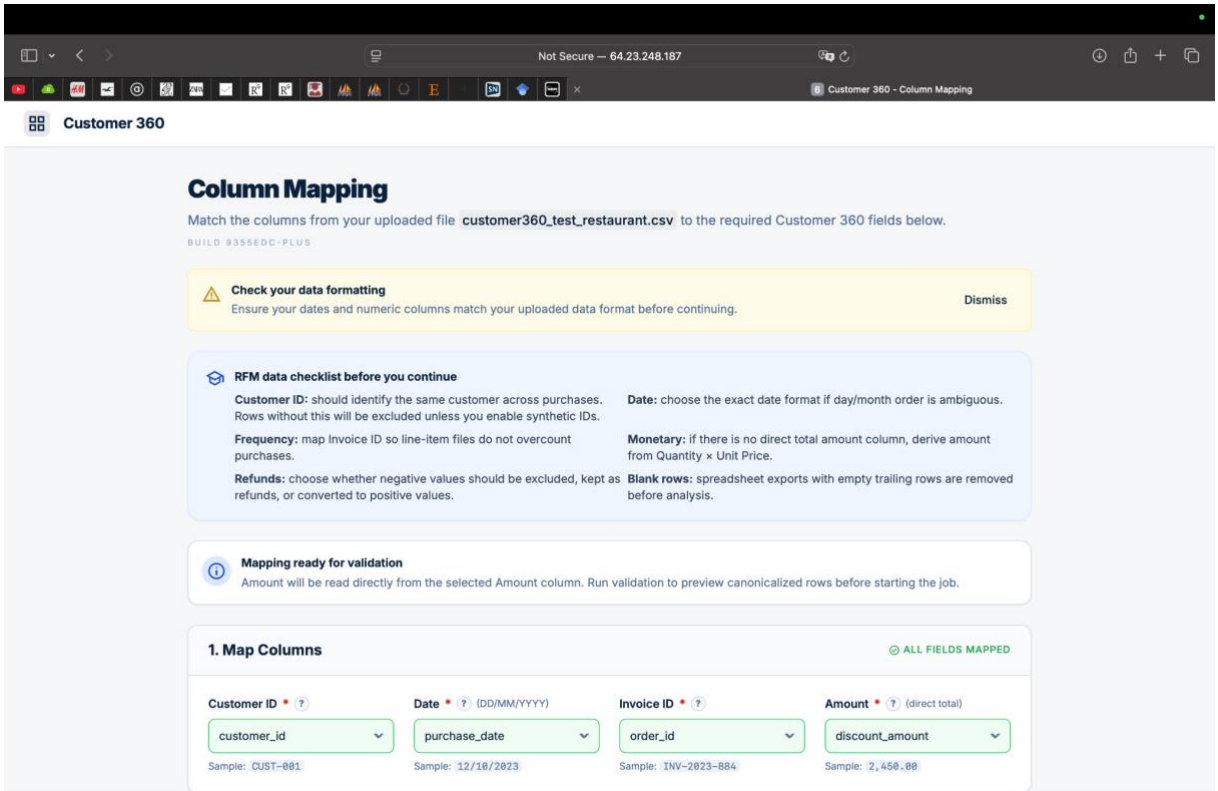


Figure 5.5: Upload page for customer transaction data.



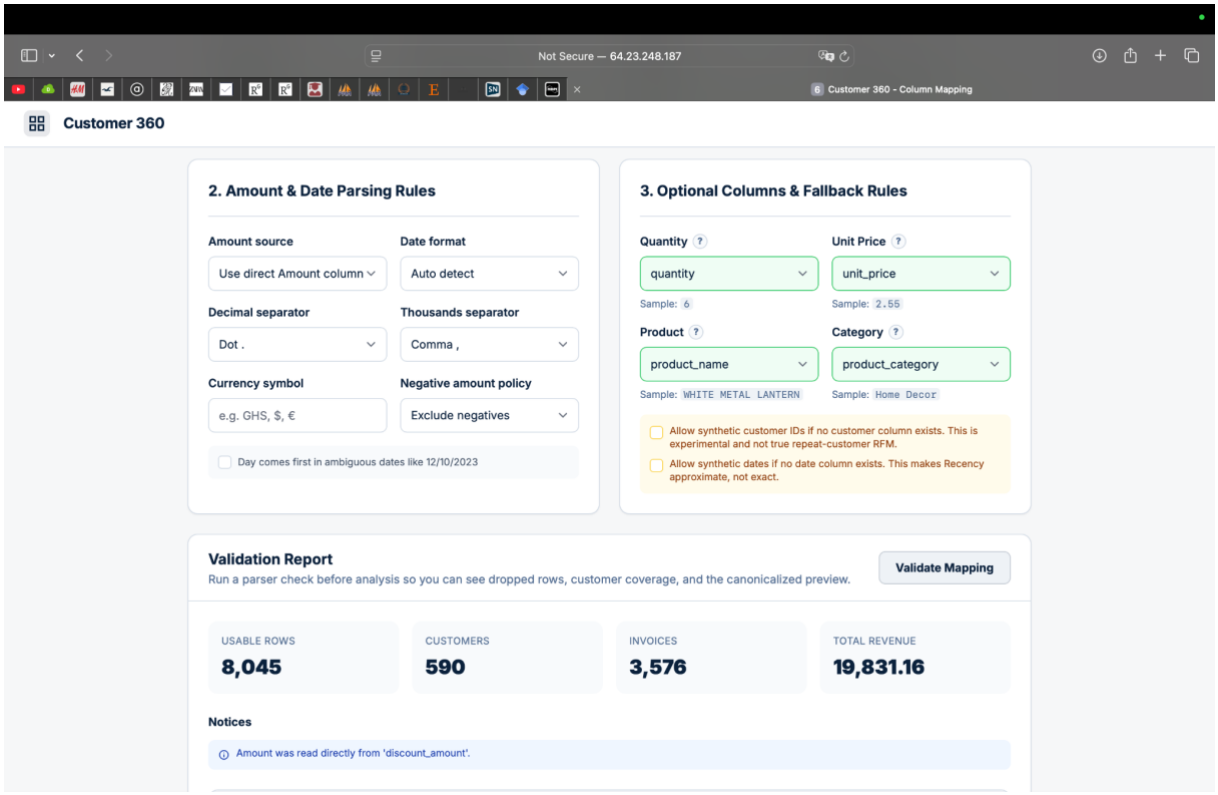


Figure 5.6,5.7: Upload page for customer transaction data.

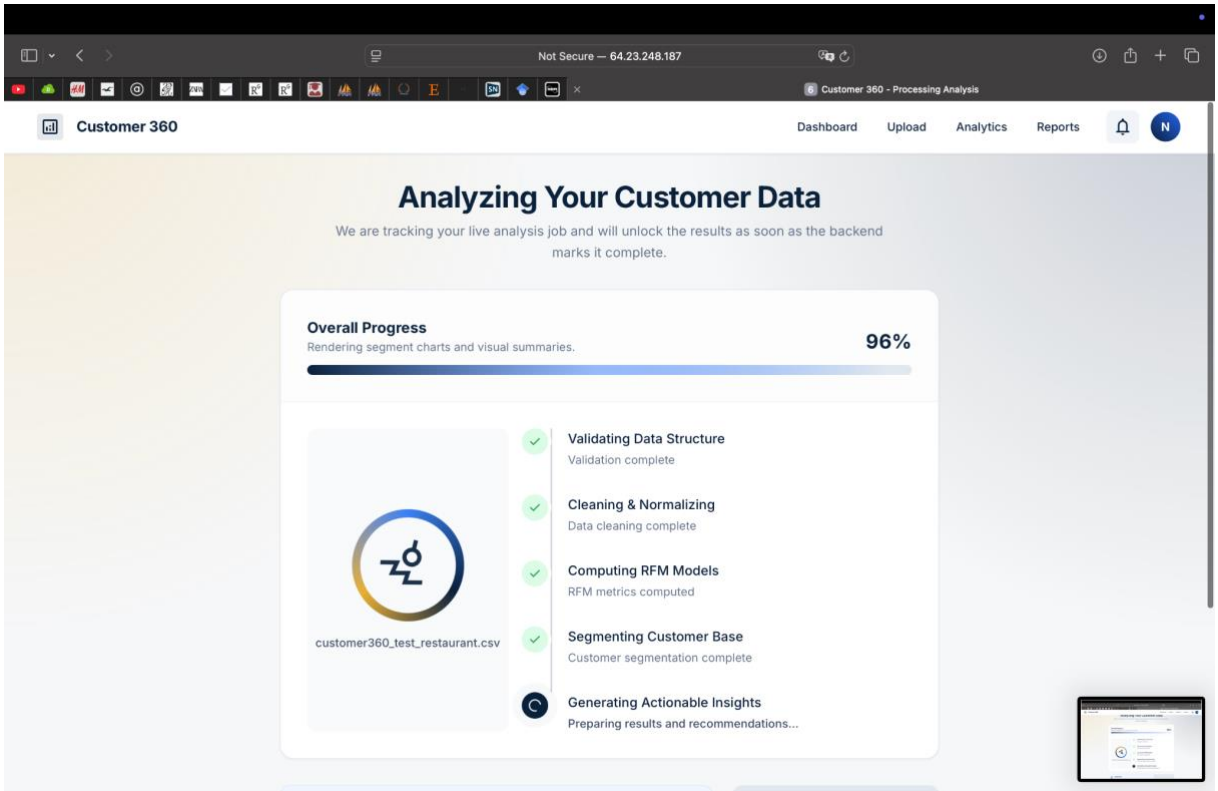
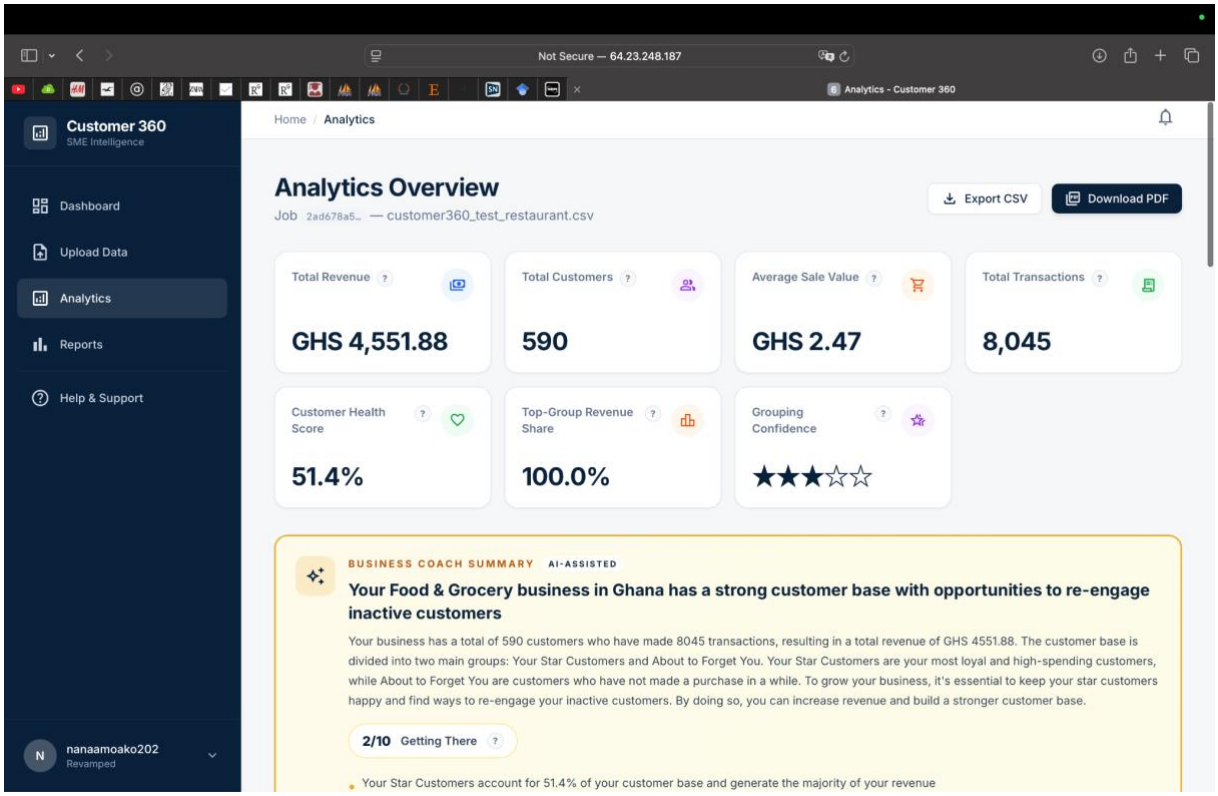
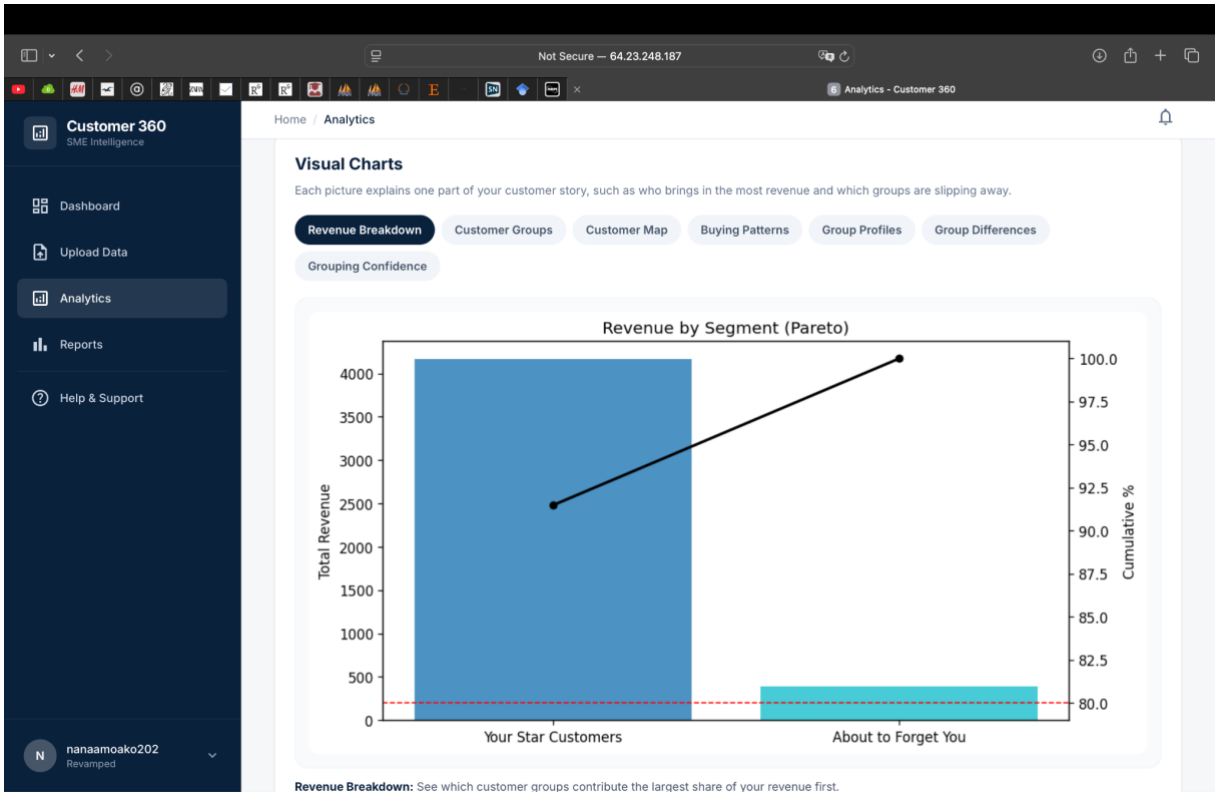
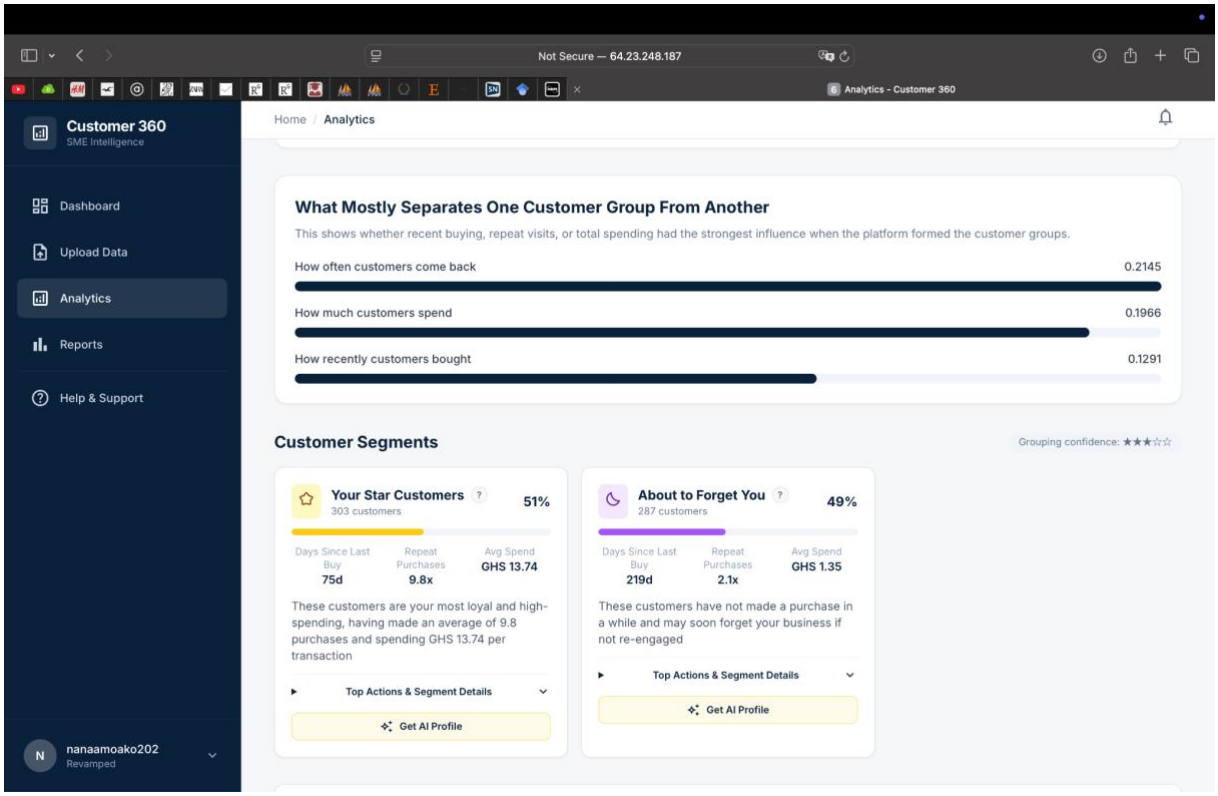


Figure 5.8: Processing page showing live job progress.


Not Secure — 64.23.248.187
Analytics - Customer 360
Customer 360
SME Intelligence
Home / Analytics
Dashboard
Upload Data
Analytics
Reports
Help & Support
Analytics Overview
Job 2ad678a5... — customer360_test_restaurant.csv
Export CSV
Download PDF
Total Revenue
GHS 4,551.88
Total Customers
590
Average Sale Value
GHS 2.47
Total Transactions
8,045
Customer Health Score
51.4%
Top-Group Revenue Share
100.0%
Grouping Confidence
BUSINESS COACH SUMMARY
AI-ASSISTED
Your Food & Grocery business in Ghana has a strong customer base with opportunities to re-engage inactive customers
Your business has a total of 590 customers who have made 8045 transactions, resulting in a total revenue of GHS 4551.88. The customer base is divided into two main groups: Your Star Customers and About to Forget You. Your Star Customers are your most loyal and high-spending customers, while About to Forget You are customers who have not made a purchase in a while. To grow your business, it's essential to keep your star customers happy and find ways to re-engage your inactive customers. By doing so, you can increase revenue and build a stronger customer base.
2/10 Getting There
Your Star Customers account for 51.4% of your customer base and generate the majority of your revenue
nanaamoako202
Revamped


Not Secure — 64.23.248.187
Analytics - Customer 360
Customer 360
SME Intelligence
Home / Analytics
Dashboard
Upload Data
Analytics
Reports
Help & Support
Visual Charts
Each picture explains one part of your customer story, such as who brings in the most revenue and which groups are slipping away.
Revenue Breakdown
Customer Groups
Customer Map
Buying Patterns
Group Profiles
Group Differences
Grouping Confidence
Revenue by Segment (Pareto)
Total Revenue
Cumulative %
0
500
1000
1500
2000
2500
3000
3500
4000
80.0
82.5
85.0
87.5
90.0
92.5
95.0
97.5
100.0
Your Star Customers
About to Forget You
Revenue Breakdown: See which customer groups contribute the largest share of your revenue first.
nanaamoako202
Revamped



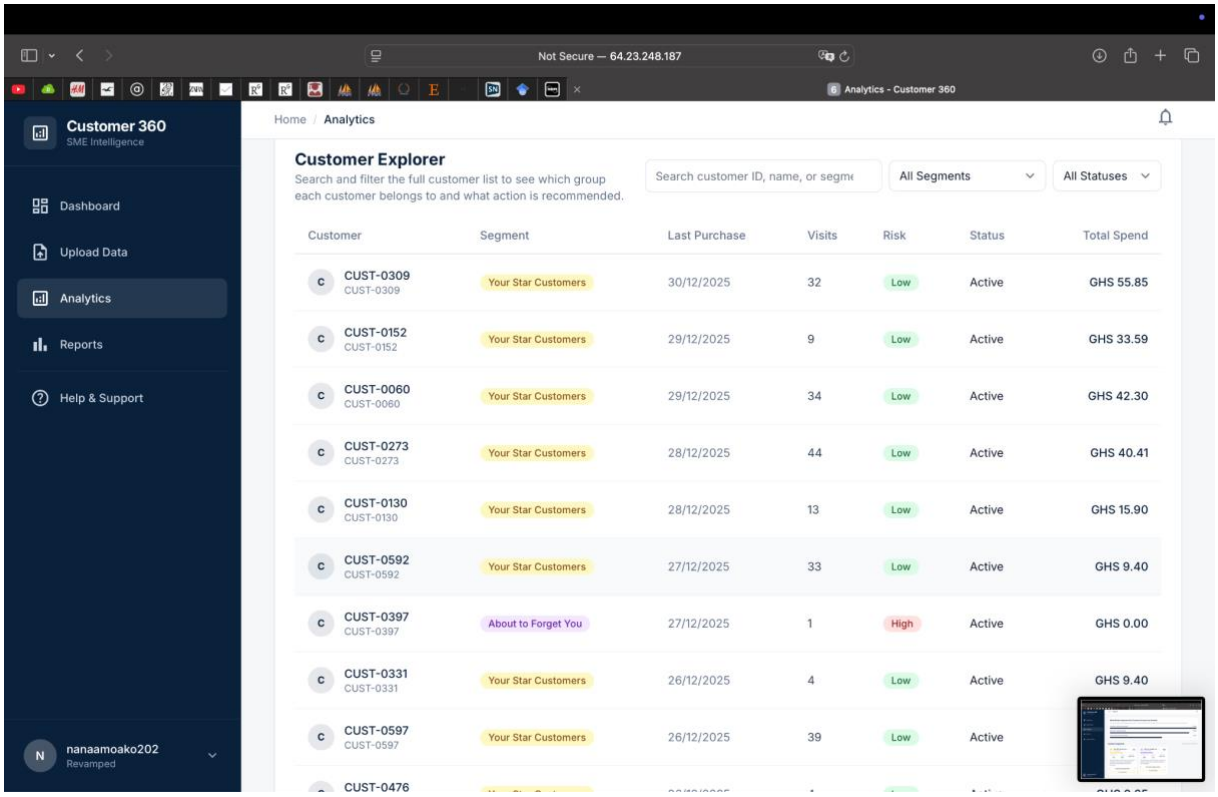


Figure 5.9,5.10,5.11,5.12 : Analytics dashboard showing completed segmentation results.

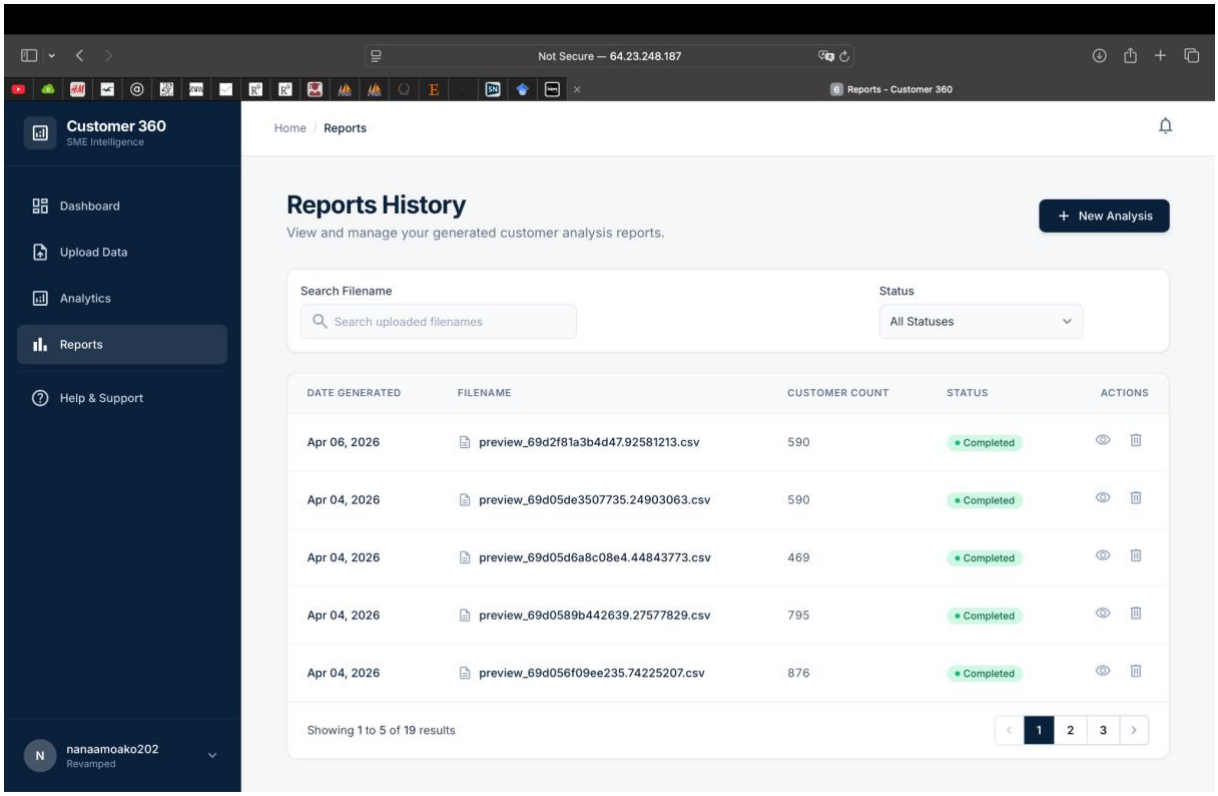


Figure 5.13: Reports page showing historical analysis runs.

5.4 User Acceptance Testing

User acceptance testing assessed whether the delivered workflow and outputs met the expectations SME stakeholders expressed during requirement gathering. The interviews and observations presented earlier in this report indicated that users expected a simple upload process, support for disorganized or informal spreadsheet records, understandable customer groupings, practical marketing recommendations, and downloadable outputs they could revisit later. The final system therefore supports a guided upload and mapping process, plain-language customer-group labels, business-facing recommendations, downloadable PDF reporting, and segmented CSV export. In practical terms, the acceptance question was whether an SME user

could upload data, complete the mapping stage, receive understandable results, and download reusable outputs without needing technical knowledge of the analytical model.

Table 5.5: User Acceptance Testing Outcomes

User Acceptance Criterion	**Observed Outcome**
Simple workflow	Achieved through a guided page-by-page process
Flexible data input	Achieved through upload preview and column mapping
Understandable outputs	Achieved through plain-language labels and summaries
Actionable value	Achieved through segment descriptions and recommendations
Reusable outputs	Achieved through PDF and CSV export

5.5 Results and Analysis

The testing results confirm that Customer360 operates as a completely deployed prototype. The analytical core passed all automated component tests, the live workflow executed successfully from file upload through to output retrieval, and deployment checks verified that the hosted architecture is functioning correctly. A full end-to-end test was run on the Austin Apparel Order History Dataset using the current pipeline.

Table 5.6: User Acceptance Testing Outcomes

Evaluation Metric	**Result**
Input transactions processed	86
Customers grouped	68
Final number of groups	2
Selected clustering method	K-Means
Silhouette Score	0.695
Full pipeline runtime	1.97 seconds

5.5.1 Issues identified and resolved during testing

Testing also uncovered several problems that code review alone would have missed. These included frontend display issues, others were backend logic errors that affected the accuracy of outputs. Each issue was fixed and re-tested before final deployment. These corrections show that the prototype was improved through actual testing rather than assumption. The specific issues and fixes are described below.

Table 5.7: Important issues discovered and resolved

Issue Identified	**Effect on the System**	**Resolution**
Incorrect backend target in the deployed PHP layer	Requests were sent to the wrong backend	PHP routing was updated to point to localhost FastAPI
Missing or incorrect server virtual environment	Backend service became unstable	The server virtual environment was recreated and the service configuration was corrected
Mapping-page deployment mismatch	Users could not continue after validation	The mapping route and frontend logic were corrected
Mapping-page JavaScript null-node errors	Validation and continue actions failed	Client-side logic was strengthened
PDF style conflict	Report generation failed	The report style configuration was corrected
Currency issue	The PDF showed poor currency formatting	Currency output was standardized to GHS
Result serialization issue	Completed jobs could fail at the output stage	Safe serialization and sanitization were added

Zero-valued executive summary in the report	Report content became misleading	Summary generation logic was corrected
Redundant segment columns in export	Reduced the quality of the user-facing CSV	Export sanitization was added

5.6 Summary

Customer360 performed successfully as a working customer segmentation prototype. The analytical backend passed all 53 automated tests on the current codebase, and the deployed workflow ran correctly from CSV upload through to PDF report generation and CSV export, confirming that functional requirements were met at both component and system level.

The system presents customer groups, recommendations, and reports in plain business language, making outputs accessible to SME owners without technical backgrounds. Testing validated analytical correctness, workflow integration, deployment stability, and output quality, and also led to the correction of several issues discovered during validation.

Chapter 6: Conclusions and Recommendations

6.1 Project Summary

Customer360 is a customer segmentation platform that uses machine learning to help small and medium-sized enterprises in Ghana. Many SMEs collect transaction records but do not have an accessible way to convert those records into usable customer insight. Customer decisions are often based on intuition, memory, or informal record-keeping rather than on structured analysis. Customer360 was built to address that gap. It is a web application with a PHP frontend, a FastAPI backend, and a MySQL metadata layer that takes uploaded transaction records and produces segmentation results in language a business owner can act on.

6.2 Assessment of the Project Objectives

Table 6.1:Summary of Project Objectives Achieved

Objective	**Degree Achieved**	**Summary of Outcome**
OBJ 1: Data collection and preparation	Achieved	The system supports CSV upload, file preview, column mapping, parser-rule validation, data cleaning, amount derivation where needed, and customer-level preprocessing suited to SME transaction data
OBJ 2: Model development and deployment	Achieved	The pipeline computes RFM features, evaluates candidate cluster counts, compares K-Means, Gaussian Mixture Model, and Hierarchical Clustering, and selects the final method automatically
OBJ 3: Evaluation and validation	Achieved	The system uses internal clustering metrics and automated testing to validate the analytical pipeline and deployed workflow, although broader repeated-

		run statistical evaluation and more formal validation can be extended in future work
OBJ 4: Explainability	Achieved	The system includes surrogate-model explainability through feature-importance summaries and related dashboard and report outputs, allowing users to understand the factors influencing grouping
OBJ 5: System integration and recommendations	Achieved	The project delivered a web application for SME users, retained a notebook-based analytical workflow for development and research, and provided grouped outputs with practical recommendations and downloadable reports

6.3 Recommendations

To guide future development of Customer360, the following areas are recommended:

- Improve data-quality support during upload. The most significant practical challenge is the quality of SME source data. Stronger templates, clearer field guidance, and more helpful validation feedback would likely improve the reliability of outputs more than adding additional analytical features.
- Introduce follow-up action tracking. Although the system provides recommendations, it does not currently capture whether businesses acted on them or whether those actions produced useful results. Adding a simple follow-up tracking feature would make the system more useful in practice.
- Support richer business-context fields. The inclusion of optional fields such as region, sales channel, product category, and customer type would make the segmentation

outputs more relevant for business decisions, especially in areas such as branch planning and targeted campaigns.

6.4 Limitations and Challenges

Main limitations and challenges of the project included the following:

1. Early deployment instability

In the earlier stages of deployment, the backend was not always consistently reachable in the intended live environment. This was mainly due to routing problems and server-environment issues, including an incorrect backend target and problems with the configured virtual environment.

2. Output-stage failures

Some analysis jobs failed late in the pipeline, not because of problems with the clustering logic, but because of issues at the result-generation stage. These included JSON serialisation problems with NumPy types, zero-valued executive summary fields, PDF style-name conflicts, currency rendering problems, and a temporary issue with chart embedding.

3.Scope limitation.

The system is designed for CSV-based transactional datasets that can be mapped into the required RFM fields. It does not yet support universal ingestion of every SME data format.

4.Unclean SME data.

Real uploaded files were not clean enough to analyse directly. This led to the addition of the preview, column-mapping, and validation flow, which was not part of the original plan but became necessary after observing actual SME datasets.

6.5 Future Work

The following areas are recommended for future development of Customer360:

1. **Improve scalability for larger datasets:** Optimize the system to handle larger datasets, increased user concurrency, and more frequent analysis runs by implementing queueing, caching, and performance tuning techniques.

2. **Customer history comparison over time.** Allowing businesses to compare segmentation results across different time periods would show whether customers are improving, declining, or remaining stable, giving a clearer picture of how the customer base is evolving.

3**. Improved operational maturity** : Automated data cleanup policies, stronger monitoring, audit logging, and clearer administrative tools.

4. **Scalability improvements** : If usage grows, the architecture should be refined to handle higher concurrency, larger datasets, and additional file formats beyond CSV. Long-term maintainability should also be reassessed as the platform expands.

5.**Action tracking after recommendations** : Allowing users to record which recommendations they acted on and whether those actions improved retention, repeat purchases, or revenue would connect the analysis directly to measurable business outcomes.

6. **Improve Deployment Architecture:** While the current colocated Apache, PHP, and FastAPI setup was appropriate for the prototype, separating the frontend and backend into independently deployed services would improve scalability, maintainability, and infrastructure flexibility in future iterations.

6.6 Conclusion

Customer360 was developed to provide a tool for Ghanaian SMEs that needed to transform transaction records into structured customer insight without needing programming knowledge. The final system includes data ingestion, preprocessing, clustering, explainability, reporting, and export. Despite the challenges faced during development and deployment, the limitations and future work outlined in this chapter provide a clear direction for continued improvement. With these upgrades, Customer360 has the potential to become a dependable and broadly applicable segmentation tool tailored to the conditions of small business operations in Ghana.

References

[1] M. Ş. Ensari and M. E. Karabay. 2014. What helps to make SMEs successful in global markets? Procedia – Social and Behavioral Sciences 150 (2014), 192–201. https://doi.org/10.1016/j.sbspro.2014.09.030

[2] OECD. 1997. Small Businesses, Job Creation and Growth: Facts, Obstacles and Best Practices. Organisation for Economic Co-operation and Development, Paris.

[3] D. Kayanula and P. Quartey. 2000. The Policy Environment for Promoting Small and Medium-Sized Enterprises in Ghana and Malawi. Institute for Development Policy and Management, University of Manchester.

[4] Ghana Chamber of SMEs. 2023. Ghana SME Sector Report 2023. GCB Bank Research Reports. Retrieved from https://www.gcbbank.com.gh/research-reports/sector-industry-reports/361-sme-sector-in-ghana-2023-v1/file

[5] S. K. Amoah and A. K. Amoah. 2018. The role of small and medium enterprises (SMEs) to employment in Ghana. International Journal of Business and Economics Research 7, 5 (2018), 151–157. https://doi.org/10.11648/j.ijber.20180705.14

[6] D. K. Agboh. 2015. Drivers and challenges of ICT adoption by SMEs in Accra Metropolis, Ghana. Journal of Technology Research 6 (2015), 1–16.

[7] L. B. Danquah. 2025. The role of artificial intelligence in enhancing decision-making in Ghanaian small and medium enterprises (SMEs). Business and Management Horizons 13, 1 (2025), 44. https://doi.org/10.5296/bmh.v13i1.23067

[8] Z. Chen. 2025. The investigation of machine learning approaches for customer segmentation. https://doi.org/10.5220/0013206800004568

[9] C. Modak, S. K. Ghosh, M. A. I. Sarkar, M. K. Sharif, M. Arif, M. Bhuiyan, and S. Devi. 2024. Customer segmentation and its impact across industries: A global study. International Journal of Business Analytics 9, 3 (2024), 78–95.

[10] E. U. Oti and M. O. Olusola. 2024. Overview of agglomerative hierarchical clustering methods. British Journal of Computer, Networking and Information Technology 7, 2 (2024), 14–23. https://doi.org/10.52589/bjcnit-cv9poogw

[11] A. J. Christy, A. Umamakeswari, L. Priyatharsini, and A. Neyaa. 2021. RFM ranking – An effective approach to customer segmentation. Journal of King Saud University – Computer and Information Sciences 33, 10 (2021), 1251–1257. https://doi.org/10.1016/j.jksuci.2018.09.004

[12] O. P. Barus, C. Nathasya, and J. J. Pangaribuan. 2023. The implementation of RFM analysis to customer profiling using K-Means clustering. Modelling, Measurement and Control C 84, 1 (2023), 35–42. https://doi.org/10.18280/mmep.100135

[13] S. Badghish and Y. A. Soomro. 2024. Artificial intelligence adoption by SMEs to achieve sustainable business performance: Application of technology–organization–environment framework. Sustainability 16, 5 (2024), 1864. https://doi.org/10.3390/su16051864

[14] C.-G. Wong, G.-K. Tong, and S.-C. Haw. 2024. Exploring customer segmentation in e-commerce using RFM analysis with clustering techniques. Journal of Telecommunications and the Digital Economy 12, 3 (2024), 97–125. https://doi.org/10.18080/jtde.v12n3.978

[15] S. M. Lundberg and S.-I. Lee. 2017. A unified approach to interpreting model predictions. In Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 4765–4774.

[16] J. N. Chukwunweike, D. Abiodun, O. Bright, and R. Taiwo. 2024. Adoption of machine learning in e-commerce and its impact on personalization and performance. Journal of E-Commerce Innovation 18, 2 (2024), 45–58.

[17] K. Tsiptsis and A. Chorianopoulos. 2011. Data Mining Techniques in CRM: Inside Customer Segmentation. John Wiley & Sons.

[18] A. M. Hughes. 1994. Strategic Database Marketing. Probus Publishing.

[19] M. Alves Gomes and T. Meisen. 2023. A review on customer segmentation methods for personalized customer targeting in e-commerce. Information Systems and e-Business Management 21 (2023), 527–570. https://doi.org/10.1007/s10257-023-00640-4

[20] J. M. John, O. Shobayo, and B. Ogunleye. 2023. An exploration of clustering algorithms for customer segmentation in the UK retail market. Analytics 2, 4 (2023), 809–823. https://doi.org/10.3390/analytics2040042

[21] P. J. Rousseeuw. 1987. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics 20 (1987), 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

[22] S. Aftab, M. Ahmad, N. Hameed, M. S. Bashir, I. Ali, and Z. Nawaz. 2023. Customer analysis using machine learning-based classification algorithms for effective segmentation using RFMT. Sensors 23, 6 (2023), 3180. https://doi.org/10.3390/s23063180

[23] C. Molnar. 2022. Interpretable Machine Learning: A Guide for Making Black Box Models Explainable (2nd ed.). Retrieved from https://christophm.github.io/interpretable-ml-book/

[24] 2025. A comparative analysis of leading clustering algorithms. International Journal of Novel Research and Development 10, 4 (2025). Retrieved from https://ijnrd.org/papers/IJNRD2504030.pdf

[25] I. Lewaa, A. Shaalan, and M. Saleh. 2023. Customer segmentation using machine learning model. Journal of Data Science and Intelligent Systems (2023). https://doi.org/10.47852/bonviewJDSIS32021293

[26] S. Wang, L. Sun, and Y. Yu. 2024. A dynamic customer segmentation approach by combining LRFMS and multivariate time series clustering. Scientific Reports 14 (2024), 17491. https://doi.org/10.1038/s41598-024-68621-2

[27] M. Naderipour and S. Hosseini Nasabian. 2025. A comprehensive RFM-A framework: Integrating age for enhanced customer segmentation. Transactions on Soft Computing 1, 1 (2025), 14–26.

[28] P. Sharma, R. Kumar, and S. Singh. 2026. A systematic literature review on explainable AI for churn prediction, customer segmentation and retention. International Journal of Data Science and Analytics (2026). https://doi.org/10.1007/s41060-025-01018-0

[29] A. Majeed et al. 2024. Challenges of SMEs adoption of big data in Africa. In Digital Transformation of SMEs in Sub-Saharan Africa. https://doi.org/10.13140/RG.2.2.29667.37924

[30] L. Butala and B. G. Mwanza. 2025. Analyzing the utilization of data analytics in supply chain decision-making among SMEs in Lusaka, Zambia. African Journal of Management and Business Research 17, 1 (2025), 475–501. https://doi.org/10.62154/ajmbr.2024.017.010528

[31] A. Wijaya, Yudhistira, et al. 2021. Davies Bouldin Index algorithm for optimizing clustering case studies mapping school facilities. TEM Journal 10, 3 (2021), 1099–1103. https://doi.org/10.18421/tem103-13

[32] M. Stawarz. 2025. Analysis of global stock market development—Integration of clustering, classification, and Shapley values. PLOS ONE 20, 6 (2025), e0326809. https://doi.org/10.1371/journal.pone.0326809

[33] P. Shu, et al. 2025. Application of cluster analysis based on SHAP values in hemodialysis patients using arteriovenous fistula. International Journal of General Medicine 18 (2025), 5475–5489. https://doi.org/10.2147/ijgm.s533419

[34] K. Abrokwah-Larbi and Y. Awuku-Larbi. 2024. The impact of artificial intelligence in marketing on the performance of business: Evidence from SMEs in an emerging economy. Sustainability 16, 5 (2024), 1864. https://doi.org/10.3390/su16051864

APPENDIX

Appendix 1 : AI Use Declaration

Project title: CUSTOMER SEGMENTATION USING MACHINE LEARNING FOR SMALL AND MEDIUM GHANAIAN ENTERPRISES.

Student(s): Akua Birago Oduro

Student ID(s): 87012026

Degree Program (CS/MIS): MIS

Supervisor(s): Professor Robert Sowah

Project type: Applied(Individual)

"I/we declare that the above accurately captures our AI tool usage. We accept responsibility for the integrity and accuracy of the work."

Student signature(s): **Date:** 7th April, 2026

Supervisor acknowledgement: _____ **Date:** _____

Appendix 2 : AI Tools and Usage

Table 7.1: Summary of AI Tools and Their Usage in the Project

AI Tool Used	Description of AI Use	Purpose
GitHub Copilot (Claude Haiku)	Reviewed initial backend implementation ideas and suggested boilerplate code for FastAPI endpoints, test structures, and database queries, which were manually revised and integrated.	To support backend development and reduce time spent on repetitive coding tasks
GitHub Copilot	Reviewed parts of the project report and suggested improvements to structure and clarity.	To improve the readability and organisation of the report
Codex	Reviewed SQL queries and suggested refinements.	To help improve the correctness and structure of database queries
GitHub Copilot	Suggested an initial structure for pytest unit and component tests, which	To support the organisation of backend testing

	were then completed and verified manually.	
Antigravity (Google DeepMind)	Helped identify relevant academic sources and suggested possible additions to the analysis pipeline and Chapter 4 documentation.	To support the literature review, strengthen the analysis, and refine documentation
Claude (Anthropic)	Reviewed PlantUML Gantt chart syntax, suggested formatting improvements, and assisted with export formatting.	To improve the correctness and presentation of the project timeline materials
ChatGPT (OpenAI)	Reviewed research notes, suggested clearer wording for selected sections, and helped refine report structure and code organisation ideas.	To improve clarity, structure, and presentation

Appendix 3: Coding Architecture

3.1: System Code Architecture

Customer360 Finalized Coding Architecture

Table 7.2: Final System Coding Architecture and File Structure

File Name	One-Line Description	Contribution
index.php	Root public entry that redirects /customer360/ to the intended frontend landing page.	Me only
frontend/landing.php	Main public landing page describing the shipped Customer360 workflow.	Me only
frontend/register.php	User registration page.	Me Only
frontend/signin.php	User sign-in page.	Me Only
frontend/dashboard.php	Signed-in dashboard overview page.	Me only
frontend/upload.php	CSV upload page and entry point into the guided analysis workflow.	Me only
frontend/column-mapping.php	Preview, mapping, and validation page for uploaded transaction files.	AI + Me
frontend/processing.php	Live progress-tracking page for submitted analysis jobs.	AI + Me

frontend/analytics.php	Primary completed-results dashboard for segmentation outputs.	AI + Me
frontend/reports.php	Historical jobs and report access page.	AI + Me
frontend/help.php	User help and support page.	Me only
frontend/api/config.php	Shared PHP configuration and helper functions for backend communication .	Me only
frontend/api/auth.php	PHP authentication proxy and session-check endpoint.	AI Only
frontend/api/upload.php	Upload and preview proxy to the FastAPI backend.	AI + Me
frontend/api/mapping.php	Mapping validation and job-submission proxy endpoint.	AI + Me
frontend/api/process.php	Proxy for progress polling, chart retrieval, report download, and CSV export.	AI + Me
frontend/api/groq-insight.php	Segment-level AI insight endpoint used by the analytics page profiler.	AI Only
frontend/css/styles.css	Shared stylesheet for	Me Only

	frontend presentation.	
frontend/assets/js/ai_profiler.js	Client-side logic for per-segment AI profile interactions.	AI + Me
frontend/assets/images/landing-hero-analytics.png	Selected hero image used on the landing page.	AI + Me
frontend/templates/rfm_customer_transactions_template.csv	Final user-facing CSV template aligned with the mapping workflow.	AI + Me
backend/app/main.py	FastAPI application entry point and router registration.	AI + Me
backend/app/config.py	Environment configuration, limits, paths, and Groq settings.	AI + Me
backend/app/database.py	Database setup, sessions, and runtime schema hardening.	AI + Me
backend/app/models.py	Database models for users and jobs.	AI + Me
backend/app/schemas.py	Request and response schemas used by the API.	AI + Me
backend/app/auth.py	Authentication helpers for password hashing and token handling.	AI + Me
backend/app/storage.py	Storage helper functions for	AI + Me

	uploads and outputs.	
backend/app/report.py	PDF report generation module.	AI + Me
backend/app/routes/auth.py	Authentication routes for registration and sign-in.	AI + Me
backend/app/routes/jobs.py	Core job routes for preview, upload, validation, status, results, reports, and exports.	AI + Me
backend/app/analytics/preprocessing.py	Data cleaning, parsing, validation, and preview preparation logic.	AI + Me
backend/app/analytics/rfm.py	Customer-level RFM feature engineering.	AI + Me
backend/app/analytics/clustering.py	Cluster selection, metric scoring, and algorithm execution.	AI + Me
backend/app/analytics/segmentation.py	Business-facing segment naming, descriptions, and actions.	AI + Me
backend/app/analytics/pipeline.py	Final end-to-end analytical pipeline orchestration.	AI + Me
backend/app/analytics/groq_analysis.py	Optional Groq narrative generation and rate limiting.	AI Only
backend/tests/test_preprocessing.py	Automated preprocessing verification.	AI + Me

backend/tests/test_rfm.py	Automated RFM verification.	AI + Me
backend/tests/test_clustering.py	Automated clustering verification.	AI + Me
backend/tests/test_api.py	Automated API verification.	AI + Me
backend/requirements.txt	Backend dependency manifest.	AI Only
backend/pytest.ini	Pytest configuration.	AI Only
backend/database/init.sql	Base database initialization script.	AI Only
backend/database/customer360.sql	Current exported database schema.	Me Only
backend/database/2026-04-01_job_schema_hardening.sql	Jobs-table schema hardening migration.	AI Only

Appendix 4: Project Management Gantt Chart

The figure below shows the Gantt chart used to plan and track the major activities of the project.

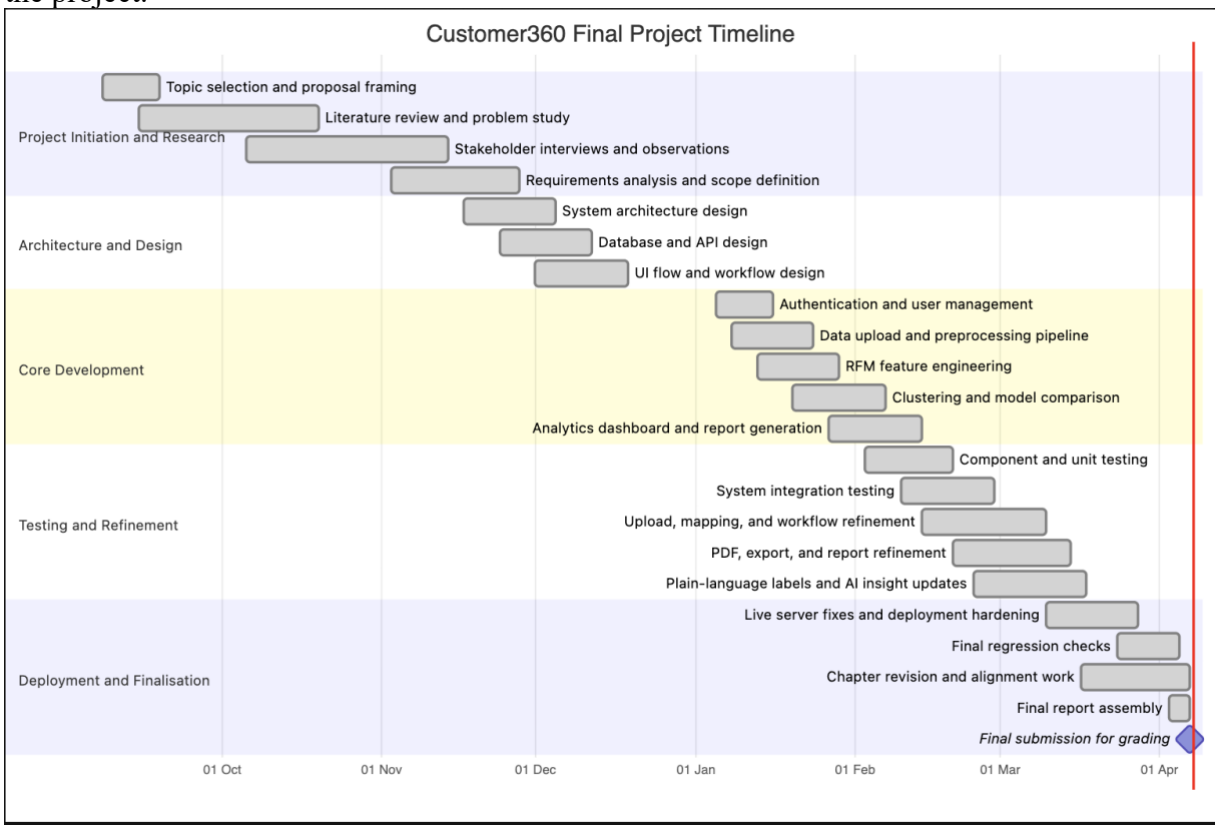


Figure 5.13: Final project-management Gantt chart for Customer360 showing the completed project timeline from project initiation to final submission.

Appendix 5 : Customer360 Semi-Structured Interview Guide : SME Stakeholders

Section	Questions
A. Business Background	1. Tell me about your business — what do you sell or offer, and who are your typical customers?
	2. How long have you been running this business?
	3. Roughly how many customers do you deal with in a typical month?
	4. What does a normal day look like when it comes to handling customer orders or inquiries?
B. Current Customer Data Practices	1. How do you currently keep track of your customers and their purchases?
	2. Walk me through where that information is stored — is it in a notebook, a phone, spreadsheets, or something else?
	3. What kind of customer information do you actually have access to right now?
	4. How often do you look back at customer records when making decisions about marketing or promotions?
	5. What makes it difficult to keep good customer records?
C. Customer Segmentation & Marketing	1. Do you currently group or categorize your customers in any way? If so, how?
	2. How do you decide which customers to focus on when running a promotion or campaign?
	3. Can you think of a time when you wished you knew more about your customers' buying behaviour? What happened?
	4. What is the biggest frustration you face when trying to understand your customers?
	5. If you could know three things about your customers that you don't know right now, what would they be?

D. Reaction to Customer360 Concept	1. *(After brief demo/description of Customer360)* What is your first impression of this kind of tool?
	2. Does the idea of uploading a CSV file and mapping columns feel manageable, or does that sound complicated? Why?
	3. If the system grouped your customers into segments like "Star Customers," "Sleeping Customers," or "Danger Zone" would those labels make sense to you? What would you do with that information?
	4. What kind of output would be most useful to you ;charts, written summaries, downloadable reports, or something else?
	5. How would you feel about a system that explains *why* customers were grouped a certain way, not just what the groups are?
E. Trust, Privacy & Concerns	1. How would you feel about uploading your customer data to an online platform?
	2. What concerns, if any, would you have about a machine learning tool making decisions about your customers?
	3. Would you want the system to delete your data after processing, or keep it so you can come back later?
	4. What would make you trust this kind of tool enough to use it regularly?
F. Practical Use & Adoption	1. Can you imagine using a tool like this in your day-to-day business? Where would it fit?
	2. What would prevent you from using it?
	3. Who else in your business or network would benefit from something like this?
	4. Would you prefer to use this on a phone, a laptop, or either?
G. Improvements & Wishlist	1. If you could change one thing about what I've described, what would it be?
	2. What features are missing that would make this actually useful for a business like yours?
	3. Is there anything about the way customer tools currently work that frustrates you and that this should avoid?

H. Final Thoughts	1. Is there anything else you want to share about how you manage customers or what you'd need from a tool like this?
	2. Would you be willing to test an improved version in the future?